Université de Bretagne Occidentale
Faculté des Sciences et Techniques
Département Informatique

Mémoire de fin d'études
pour l'obtention du diplôme de Master en Informatique
parcours Logiciels pour les Systèmes Embarqués

Caractérisation de la consommation énergétique des LLM sur des périphériques en périphérie

---

Réalisé par :
KEDADSA Islam Chakib

Encadré par :
M. Boukhobza Jalil
M. Lannurien Vincent
Mme. Bouzouad Meriem

Organisme d'accueil :
École Nationale Supérieure de Techniques Avancées, Brest, France

2025 / 2026

Remerciements

Je souhaite tout d'abord adresser mes sincères remerciements à toutes les personnes qui m'ont accompagné et soutenu tout au long de ce stage et de la réalisation de ce mémoire.

Je remercie tout particulièrement **M. Jalil BOUKHOBZA** pour la qualité de son encadrement, ses orientations, sa disponibilité et la confiance qu'il m'a accordée durant ce travail. Ses conseils et son suivi ont été précieux à chaque étape du projet.

J'adresse également mes chaleureux remerciements à **Mme Meriem BOUZOUAD** et à **M. Vincent LANNURIEN** pour leur disponibilité, leurs conseils, leurs échanges et leur accompagnement constant tout au long du stage. Leur implication a largement contribué au bon déroulement de ce travail.

Je tiens aussi à remercier **M. Stéphane RUBINI**, tuteur de l'UBO, pour son suivi, son attention et son accompagnement au cours de cette expérience.

Je remercie tout particulièrement **Lina**, avec qui j'ai eu la chance de partager cette expérience à l'UBO puis à ENSTA Bretagne. Notre collaboration, nos nombreux échanges et l'entraide tout au long du projet ont été particulièrement précieux et ont contribué à l'avancement de ce travail.

Mes remerciements vont également au **jury** pour le temps consacré à l'évaluation de ce mémoire, ainsi qu'à l'**équipe pédagogique et administrative de l'UBO** pour la qualité de la formation et de l'accompagnement proposés.

Enfin, j'exprime toute ma gratitude à ma **famille** pour son soutien, ses encouragements et sa présence tout au long de mon parcours. Je remercie également mes **amis** et toutes les personnes qui, par leur aide ou leurs encouragements, ont contribué de près ou de loin à l'aboutissement de ce travail.

Résumé

Le déploiement croissant des modèles de langage de grande taille (LLM) sur des plateformes *Edge* soulève des enjeux importants de performance et de consommation énergétique en raison de leurs ressources de calcul, de mémoire et de puissance limitées. Les travaux existants étudient plusieurs facteurs susceptibles d'influencer cette consommation, tels que la charge, le modèle ou la configuration du système. Cependant, la configuration matérielle, notamment le choix entre CPU et GPU, reste encore peu étudiée et modélisée, malgré les avantages que peut représenter l'exploitation de ces deux ressources.

Ce travail vise ainsi à caractériser et à modéliser l'écart de performance et d'efficacité énergétique entre les exécutions CPU et GPU sur une plateforme *Edge* en fonction de paramètres applicatifs et système. Une méthodologie basée sur les *Time Rooflines* et les *Energy Rooflines* est mise en place sur une NVIDIA Jetson Orin Nano Super. Dix configurations système sont étudiées selon une approche *One Factor At a Time*, en faisant varier la fréquence CPU, le nombre de cœurs CPU, la fréquence GPU, le nombre de TPC GPU et la fréquence mémoire. L'inférence de Qwen2.5-0.5B avec `llama.cpp` est ensuite caractérisée en distinguant les phases de *prefill* et de *decode*, dont l'intensité arithmétique est estimée à partir du graphe d'exécution. Les mesures énergétiques sont enfin collectées simultanément à partir des capteurs internes de la plateforme et d'un analyseur de puissance externe.

Les résultats montrent que le GPU conserve des plafonds de performance et d'efficacité énergétique supérieurs sur la plage étudiée, mais que l'écart avec le CPU diminue fortement pour les charges de faible intensité arithmétique. L'augmentation du nombre de cœurs CPU produit le rapprochement global le plus important. Les charges LLM réelles suivent globalement les variations mises en évidence par les Rooflines, avec toutefois des écarts CPU/GPU plus faibles, en particulier pendant le *decode*, où une quasi-parité énergétique est atteinte dans certaines configurations. Enfin, malgré un écart moyen d'environ 21,2 % sur les valeurs énergétiques absolues, les deux méthodes de mesure conduisent à des variations relatives et à des classements très proches. Ces résultats constituent une base pour de futurs travaux d'optimisation visant à adapter la ressource d'exécution et la configuration système aux caractéristiques de la charge et aux contraintes énergétiques.

Table des matières

Remerciements **i**

Résumé **ii**

Acronymes **vii**

Introduction générale **1**

1 Concepts fondamentaux de l'inférence LLM sur *Edge* **4**
1.1 Introduction 4
1.2 Architecture Transformer et grands modèles de langage 4
1.2.1 Fondements du Transformer 4
1.3 Inférence des modèles de langage 6
1.3.1 Déroulement général d'une requête 6
1.3.2 Phase de *prefill* 6
1.3.3 Phase de *decode* 7
1.4 Exécution des modèles de langage sur périphérie 7
1.4.1 SLM 7
1.4.2 Moteurs d'inférence 8
1.4.3 Plateformes matérielles 9
1.5 Gestion et mesure de l'énergie 9
1.5.1 Puissance, énergie et composantes de la consommation 10
1.5.2 Gestion de la puissance 10
1.5.3 Mesure de la puissance 12
1.6 Modélisation de l'énergie et des performances 13
1.6.1 *Time Roofline* 13
1.6.2 *Energy Roofline* 14
1.7 Conclusion 15

2 État de l'art **16**
2.1 Introduction 16
2.2 Caractérisation et modélisation énergétique de l'inférence des LLMs 16
2.2.1 Caractérisation des facteurs d'inférence 16
2.2.2 Modélisation de la consommation énergétique 18
2.2.3 Fiabilité de la mesure énergétique 19
2.3 Optimisation énergétique de l'inférence des LLMs 19

2.3.1 Optimisations applicatives et logicielles . . . 19
2.3.2 Optimisations du système . . . 19
2.3.3 Optimisations conjointes . . . 20
2.4 Synthèse et positionnement du travail . . . 20
2.5 Conclusion . . . 21

3 Méthodologie de caractérisation 24
3.1 Introduction . . . 24
3.2 Questions de recherche . . . 24
3.3 Démarche de caractérisation . . . 25
3.3.1 Étape 1 : Modélisation *Roofline* . . . 25
3.3.2 Étape 2 : Exécution et modélisation des charges LLM . . . 25
3.3.3 Étape 3 : Comparaison des mesures énergétiques . . . 26
3.4 Mise en œuvre expérimentale . . . 26
3.4.1 Environnement expérimental commun . . . 26
3.4.2 Campagne 1 : Construction des *Rooflines* . . . 28
3.4.3 Campagne 2 : Caractérisation des charges de travail LLM . . . 29
3.5 Conclusion . . . 31

4 Résultats et discussion 32
4.1 Introduction . . . 32
4.2 Q1 – Écart entre CPU et GPU selon l'intensité arithmétique et les configurations système . . . 32
4.2.1 Influence des paramètres système . . . 35
4.2.2 Réponse à Q1 . . . 36
4.3 Q2 – Transposition des écarts CPU/GPU des Rooflines aux charges de travail LLM . . . 37
4.3.1 Réponse à Q2 . . . 40
4.4 Q3 – Influence de l'instrument de mesure sur les conclusions énergétiques . . . 40
4.4.1 Réponse à Q3 . . . 42
4.5 Synthèse du chapitre . . . 42
4.6 Conclusion . . . 43

Conclusion générale 44

Table des figures

1.1 Architecture originale du Transformer proposée par Vaswani et al. [37]. . . . 5
1.2 Représentation schématique du *Time Roofline* et de l'*Energy Roofline*. 15

3.1 Protocole commun d'une acquisition expérimentale. 28

4.1 Écarts GPU/CPU et paramètres des Rooflines selon les configurations OFAT. Les lignes pointillées des panneaux (c)–(f) indiquent les valeurs de référence B0. 33
4.2 Fraction des capacités théoriques atteinte par les plafonds empiriques. Les lignes pointillées indiquent les valeurs de référence B0. 34
4.3 Positionnement des charges LLM sur les Rooflines et comparaison des écarts CPU/GPU. 38
4.4 Évolution de l'écart GPU/CPU des phases de *prefill* et de *decode* selon les configurations système. 41
4.5 Comparaison des mesures énergétiques d'Alumet et du Power Analyzer. . . . 41

Liste des tableaux

2.1 Positionnement des travaux selon les dimensions étudiées. 22
2.2 Positionnement des travaux sur l'inférence LLM. 23

3.1 Configurations système utilisées pour l'étude OFAT. 27
3.2 Charges de travail LLM utilisées pour la caractérisation. 29
3.3 Règles de calcul de W, Q et de l'intensité arithmétique pour les opérations observées dans les graphes `llama.cpp`. 30
3.4 Quantité de calcul, volume logique de données et intensité arithmétique des charges de travail LLM étudiées. 31
3.5 Comparaison entre l'estimation issue du graphe et les mesures NCU pour P512. 31

Acronymes

CMOS *Complementary Metal-Oxide-Semiconductor*. 10
CPU *Central Processing Unit*. 2, 9, 11, 13, 16, 24–29, 32, 34–36, 39, 40, 42–45

DRAM *Dynamic Random-Access Memory*. 9, 11, 31, 35, 36
DSP *Digital Signal Processor*. 9
DVFS *Dynamic Voltage and Frequency Scaling*. 10, 11

EMC *External Memory Controller*. 11, 26, 27, 36

GPU *Graphics Processing Unit*. 2, 9, 11, 13, 16, 24–29, 31, 32, 34–36, 39, 40, 43–45

LLM *Large Language Model*. 1–4, 6, 7, 16, 18, 24–26, 28, 29, 39, 40, 43, 44
LSTM *Long Short-Term Memory*. 5

NPU *Neural Processing Unit*. 9

RAPL *Running Average Power Limit*. 12
RNN *Recurrent Neural Network*. 5

SLM *Small Language Model*. 7, 17

TPC *Texture Processing Cluster*. 11, 26, 27, 36

Introduction générale

Contexte général

L'essor de l'intelligence artificielle générative a profondément modifié les usages et les paradigmes de l'apprentissage automatique [33]. Les modèles de langage de grande taille (*Large Language Models*, *Large Language Models* (LLMs)), initialement associés à des tâches de traitement et de génération de texte, sont désormais intégrés à des services interactifs, des assistants intelligents et des applications capables de traiter des données de différentes natures. Leur taille augmente et leur déploiement s'étend à un nombre croissant d'applications, ce qui entraîne une multiplication des inférences, parfois exécutées de manière continue ou à grande échelle pour répondre à de nombreuses requêtes. Dès lors, le coût d'un LLM ne se limite plus aux ressources nécessaires à son entraînement ; son déploiement et, en particulier, le coût cumulé de l'inférence deviennent également des enjeux importants en termes de ressources matérielles et d'énergie.

Ces besoins ont jusqu'à présent favorisé le recours aux infrastructures *cloud* et aux centres de données, capables de fournir les importantes ressources de calcul et de mémoire nécessaires à l'exécution de ces modèles. Cette capacité de calcul repose notamment sur l'utilisation d'accélérateurs matériels performants, dont le déploiement à grande échelle contribue également à accroître les besoins énergétiques de ces infrastructures. En effet, la consommation électrique des centres de données constitue un enjeu majeur : en 2024, les centres de données ont consommé environ 415 TWh d'électricité, soit près de 1,5 % de la consommation électrique mondiale, avec une croissance moyenne d'environ 12 % par an au cours des cinq années précédentes [16]. L'Agence internationale de l'énergie souligne notamment que le développement de l'intelligence artificielle accélère le déploiement de serveurs équipés d'accélérateurs et contribue à l'augmentation de la densité de puissance de ces infrastructures.

Cependant, toutes les applications utilisant des LLMs ne se prêtent pas nécessairement à une exécution exclusivement distante. Certaines nécessitent des temps de réponse faibles, manipulent des données qui ne doivent pas quitter leur environnement de production ou doivent continuer à fonctionner avec une connectivité limitée. Ces contraintes conduisent à rapprocher une partie du calcul de l'endroit où les données sont produites et consommées, suivant le paradigme de l'*Edge Computing* [32]. Mais déplacer l'inférence du *cloud* vers l'*Edge* déplace également le problème vers des plateformes soumises à des contraintes beaucoup plus fortes. Ces plateformes doivent donc assurer des traitements suffisamment rapides tout en respectant leurs contraintes énergétiques et thermiques. Une même inférence peut par ailleurs présenter des comportements très différents selon les ressources matérielles utilisées, leur

configuration et les caractéristiques du modèle ou de la requête. Dans ce contexte, caractériser conjointement les performances et la consommation énergétique devient une étape essentielle pour comprendre dans quelles conditions l'exécution locale d'un LLM peut être efficace et pour identifier les leviers permettant de l'optimiser.

Cadre, problématique et objectifs du projet

Ces enjeux constituent le point de départ du projet mené à l'ENSTA Campus Brest, au sein du Laboratoire des Sciences et Techniques de l'Information, de la Communication et de la Connaissance (Lab-STICC), une unité de recherche pluridisciplinaire du CNRS regroupant plusieurs établissements d'enseignement supérieur et de recherche. Plus précisément, ce travail s'inscrit dans le pôle SHARP (*Software and Hardware, ARchitectures and Processes*), au sein de l'équipe SHAKER, dont les travaux portent notamment sur l'étude et l'optimisation conjointe des systèmes matériels et logiciels.

Dans ce cadre, le projet s'intéresse plus particulièrement à la caractérisation de la consommation énergétique des LLMs lors de leur exécution sur des plateformes *Edge*. Cette consommation résulte de l'interaction entre plusieurs facteurs applicatifs et paramètres système susceptibles de l'affecter [4, 22]. Elle peut également varier selon la ressource matérielle utilisée, *Central Processing Unit* (CPU) ou *Graphics Processing Unit* (GPU), ainsi que selon les différentes phases de l'inférence. L'analyse conjointe de ces deux dimensions reste encore peu abordée dans les travaux existants. À cela s'ajoute une dimension méthodologique liée à la mesure de la puissance, puisque les outils et protocoles de mesure peuvent conduire à des écarts significatifs dans les résultats obtenus [2]. Notre objectif est donc d'analyser conjointement ces dimensions afin de mieux comprendre les variations de performance et de consommation observées et d'identifier les paramètres susceptibles de guider des stratégies d'optimisation adaptées aux contraintes des plateformes *Edge*.

La problématique générale de ce travail peut ainsi être formulée comme suit :

> **Comment caractériser et modéliser la consommation énergétique de l'inférence des LLMs sur CPU et GPU au sein de plateformes Edge, afin de comprendre l'influence des facteurs applicatifs et des paramètres système et de guider des stratégies d'optimisation ?**

Pour répondre à cette problématique, le projet a été structuré autour de trois objectifs complémentaires :

1. Réaliser un état de l'art des travaux consacrés à la caractérisation et à l'optimisation énergétique de l'inférence des LLMs, en particulier sur les plateformes *Edge* ;
2. Réaliser une caractérisation énergétique de l'exécution des LLMs afin d'étudier les compromis entre performance et consommation énergétique selon les différentes phases de l'inférence, et proposer des modèles permettant de représenter le comportement des charges de travail sur CPU et GPU, en s'appuyant sur deux méthodes de mesure complémentaires : une instrumentation logicielle et un analyseur de puissance externe ;
3. Exploiter les résultats obtenus afin d'identifier des pistes d'optimisation énergétique pouvant concerner aussi bien la configuration du système, par l'adaptation des ressources matérielles et des paramètres de puissance, que les caractéristiques du modèle.

Après délimitation du périmètre de l'étude, le troisième objectif n'a pas été traité. Le travail a été recentré sur la caractérisation et la modélisation des comportements observés, qui constituent une étape préalable à cette optimisation. Les résultats obtenus pourront ainsi servir de base à de futurs travaux d'optimisation.

Organisation du document

La suite de ce mémoire est organisée comme suit :

- **Chapitre 1 : Contexte et concepts fondamentaux :** ce chapitre présente les notions nécessaires à la compréhension du travail, notamment les modèles de langage, leur processus d'inférence, les caractéristiques des plateformes *Edge* ainsi que les concepts liés aux performances et à la consommation énergétique.

- **Chapitre 2 : État de l'art :** ce chapitre présente les travaux consacrés à la caractérisation énergétique des LLMs ainsi qu'aux approches d'optimisation de leur exécution, puis se termine par le positionnement de ce travail.

- **Chapitre 3 : Méthodologie de caractérisation :** ce chapitre présente les questions de recherche et la démarche de caractérisation retenue, puis décrit sa mise en œuvre expérimentale, les différentes campagnes réalisées ainsi que les approches utilisées pour mesurer et analyser les performances et la consommation énergétique des différentes phases de l'inférence.

- **Chapitre 4 : Résultats et discussion :** ce chapitre présente et analyse les résultats obtenus au cours des différentes campagnes expérimentales afin de répondre aux trois questions de recherche.

Chapitre 1

Concepts fondamentaux de l'inférence LLM sur *Edge*

1.1 Introduction

Ce chapitre introduit les notions nécessaires à l'étude de l'inférence des modèles de langage sur des plateformes *Edge*. Il présente d'abord les principes des modèles de langage et de l'architecture Transformer, puis décrit le déroulement d'une inférence et les deux phases qui la composent. Les sections suivantes abordent les environnements matériels et logiciels de l'inférence, ainsi que les aspects liés à l'énergie et à sa modélisation, des éléments indispensables à la suite de ce mémoire.

1.2 Architecture Transformer et grands modèles de langage

Un modèle de langage est un modèle probabiliste qui apprend les régularités du langage afin d'estimer la probabilité d'un élément à partir de son contexte. Les modèles de langage modernes reposent principalement sur des réseaux de neurones entraînés sur de vastes corpus de données, et présentent des performances qui évoluent avec cette quantité de données d'entraînement, avec la taille du modèle et avec les ressources de calcul mobilisées [20]. Cette mise à l'échelle a favorisé l'émergence de modèles comportant plusieurs milliards de paramètres, désignés comme LLMs, capables d'accomplir un large éventail de tâches de compréhension et de génération du langage [6].

1.2.1 Fondements du Transformer

L'architecture Transformer, introduite par Vaswani et al. en 2017 [37], a profondément transformé la conception des modèles de langage et constitue aujourd'hui la base de nombreux LLMs. Son fonctionnement repose principalement sur un mécanisme d'attention permettant de modéliser les relations entre les différents éléments d'une séquence, tout en offrant une parallélisation plus importante du traitement par rapport aux architectures séquentielles

telles que les *Recurrent Neural Networks* (RNNs) ou les *Long Short-Term Memory networks* (LSTMs).

À partir des représentations associées aux différents éléments d'une séquence, le mécanisme d'attention permet au modèle d'évaluer leurs relations et de déterminer l'importance de chacun d'eux dans le contexte considéré. Pour chaque élément, trois projections sont calculées : une requête (*Query*, Q), une clé (*Key*, K) et une valeur (*Value*, V). Le calcul de l'attention est défini par :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{1.1}$$

où d_k représente la dimension des vecteurs de clés [37]. Le produit QK^T permet d'obtenir des scores traduisant les relations entre les éléments de la séquence. Après normalisation par la fonction *softmax*, ces scores sont utilisés pour pondérer les valeurs V et construire les nouvelles représentations.

L'architecture originale utilise plusieurs mécanismes d'attention en parallèle, dont les sorties sont combinées, appelés *têtes d'attention* (*Multi-Head Attention*). Le tout s'inscrit dans une organisation encodeur-décodeur, illustrée à la Figure 1.1. L'encodeur construit progressivement une représentation de la séquence d'entrée à travers un empilement de blocs, tandis que le décodeur exploite cette représentation pour produire la séquence de sortie. Ces blocs combinent principalement les mécanismes d'attention ainsi que des réseaux *feed-forward*, des connexions résiduelles et des opérations de normalisation.

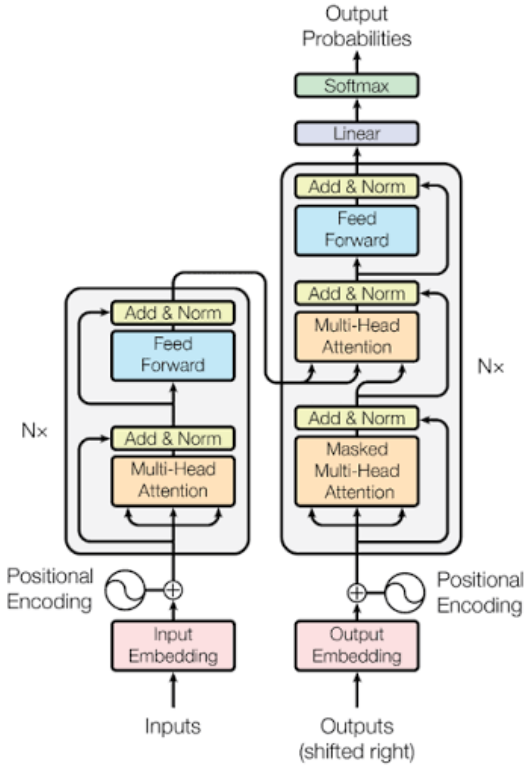


FIGURE 1.1 – Architecture originale du Transformer proposée par Vaswani et al. [37].

Pour la génération de texte, de nombreux LLMs utilisent une architecture de type *decoder-only*, notamment les modèles de la famille GPT [6]. Elle est constituée d'un empilement de blocs inspirés du décodeur du Transformer et repose sur un mécanisme d'attention causale : pour chaque position de la séquence, ce mécanisme limite l'attention aux éléments déjà disponibles dans le contexte.

Cette organisation permet une génération autorégressive : à partir d'une séquence donnée en entrée, le modèle estime la distribution de probabilité de l'élément suivant, sélectionne celui-ci puis l'ajoute au contexte. Ce processus est répété successivement jusqu'à produire la séquence de sortie souhaitée.

Le déroulement de cette génération conduit à distinguer deux phases au cours de l'inférence. Le contexte d'entrée est d'abord traité lors de la phase de *prefill*, puis les éléments de sortie sont générés successivement pendant la phase de *decode*. Ces deux phases sont détaillées dans la section suivante.

1.3 Inférence des modèles de langage

1.3.1 Déroulement général d'une requête

Lors d'une inférence, le texte fourni au modèle est d'abord transformé en une séquence de *tokens* au cours d'une étape appelée *tokenisation*. Selon le *tokenizer* utilisé, un *token* peut représenter un mot, une partie de mot, un caractère ou un symbole. Chaque *token* est associé à un identifiant appartenant au vocabulaire du modèle, puis converti en une représentation vectorielle appelée *embedding*. Une information de position est également associée à ces représentations afin de préserver l'ordre de la séquence.

Cette séquence constitue le contexte d'entrée à partir duquel la réponse est générée. Le modèle produit une distribution de probabilité sur son vocabulaire, à partir de laquelle un nouveau *token* est sélectionné selon la stratégie de génération utilisée. Celui-ci est ensuite ajouté au contexte et le processus est répété jusqu'à atteindre une condition d'arrêt ou le nombre maximal de *tokens* demandé.

L'exécution se décompose en deux phases successives, qui diffèrent par le parallélisme disponible, la nature des opérations exécutées et les données manipulées, ce qui est particulièrement important pour l'étude des performances et de la consommation énergétique : la phase de *prefill* et la phase de *decode*.

1.3.2 Phase de *prefill*

La phase de *prefill* traite simultanément l'ensemble des *tokens* du *prompt* à travers les différentes couches du modèle afin de construire leurs représentations contextuelles, en calculant, pour chaque couche d'attention, les clés (K) et les valeurs (V), qui sont conservées dans une structure appelée *KV cache* pour être réutilisées lors de la phase suivante.

Le traitement simultané de plusieurs *tokens* durant cette phase permet notamment d'effectuer les projections linéaires de l'attention et des réseaux *feed-forward* sous forme de nombreuses opérations matricielles, ce qui augmente la quantité de calcul réalisée au cours de cette phase, d'autant plus que la longueur de la séquence d'entrée est importante.

1.3.3 Phase de *decode*

La phase de *decode* génère les *tokens* de sortie de manière autorégressive. À chaque étape, le modèle traite le nouveau *token* en tenant compte des informations déjà disponibles dans le cache afin de déterminer le suivant. La génération d'une réponse repose ainsi sur une succession d'étapes de décodage, ce qui confère à cette phase un caractère intrinsèquement séquentiel.

Pour limiter les calculs redondants au cours de cette génération, les clés et les valeurs associées aux *tokens* précédemment traités sont conservées dans la structure *KV cache*. Et lorsqu'un nouveau *token* est généré, les clés et les valeurs correspondantes doivent être calculées puis ajoutées au cache, tandis que celles déjà présentes sont directement réutilisées par le mécanisme d'attention.

La taille du *KV cache* augmente ainsi avec la longueur de la séquence, le nombre de couches d'attention du modèle, la dimension des représentations stockées et la précision numérique utilisée, ce qui accroît son empreinte mémoire et entraîne une sollicitation importante de la bande passante mémoire pendant cette phase de *decode*.

1.4 Exécution des modèles de langage sur périphérie

Le déploiement de modèles de langage sur des plateformes *Edge* accompagne l'intégration croissante de capacités d'intelligence artificielle directement dans les équipements terminaux. Cependant, l'exécution de LLMs sur ces plateformes reste conditionnée par l'adéquation entre les besoins du modèle et les ressources du dispositif cible. Leur coût en calcul et en mémoire peut rapidement dépasser les capacités disponibles, ce qui motive le recours à des modèles plus compacts et à des techniques permettant de réduire les ressources nécessaires à leur exécution [38, 43].

1.4.1 SLM

La réduction des besoins du modèle constitue un premier moyen d'adapter l'inférence aux ressources disponibles sur les plateformes *Edge*. Dans cette perspective, les *Small Language Models* (SLMs) regroupent des modèles de langage de taille plus réduite, conçus pour offrir des capacités de compréhension et de génération avec des besoins plus faibles en mémoire et en calcul. Il n'existe toutefois pas de seuil universel permettant de distinguer précisément un SLM d'un LLM, cette distinction dépendant notamment du contexte d'utilisation et des ressources disponibles [25]. Plusieurs familles de modèles compacts ont ainsi été développées pour faciliter leur exécution sur des dispositifs aux ressources limitées, parmi lesquelles SmolLM2, proposé en variantes de 135 millions, 360 millions et 1,7 milliard de paramètres, ainsi que les petites variantes des familles Qwen, Phi ou Gemma [5].

L'obtention de modèles compacts ne repose pas uniquement sur la conception d'architectures de petite taille. Des techniques de compression peuvent également être utilisées pour réduire les ressources nécessaires à l'exécution d'un modèle, voire pour dériver un modèle plus compact à partir d'un modèle de plus grande taille. Parmi les approches les plus étudiées figurent la quantification, le *pruning* et la distillation de connaissances [25, 13].

Quantification. La quantification réduit la précision numérique utilisée pour représenter les paramètres du modèle et, selon les méthodes, les activations ou le *KV cache*. Des valeurs représentées initialement en FP32 ou FP16 peuvent ainsi être converties vers des formats de plus faible précision, tels que INT8 ou INT4. Cette réduction diminue l'espace nécessaire au stockage des données quantifiées ainsi que le volume de données transférées depuis la mémoire. Son effet sur les performances dépend cependant du format considéré et de sa prise en charge par le matériel et les *kernels* utilisés pour l'inférence [13].

Pruning. Le *pruning*, ou élagage, consiste à supprimer des paramètres ou des structures du modèle considérés comme peu importants. Le *pruning* non structuré agit sur des poids individuels et produit généralement des matrices creuses, tandis que le *pruning* structuré supprime des éléments complets, tels que des neurones, des têtes d'attention ou certaines dimensions des couches. La réduction obtenue en termes de coût d'exécution dépend alors de la possibilité d'exploiter efficacement cette parcimonie lors de l'inférence [13].

Distillation de connaissances. La distillation de connaissances (*Knowledge Distillation*) repose sur le transfert de connaissances d'un modèle de grande capacité, appelé modèle enseignant, vers un modèle plus compact, appelé modèle élève. L'apprentissage de ce dernier est guidé par les sorties ou certaines représentations produites par le modèle enseignant, afin d'obtenir un modèle nécessitant moins de paramètres et de ressources tout en préservant autant que possible ses capacités [13].

Ces techniques modifient ainsi la taille, la représentation numérique ou la structure du modèle afin d'en réduire les besoins lors de l'inférence. Les gains obtenus en pratique dépendent toutefois de la capacité de l'environnement d'exécution à exploiter efficacement ces transformations. Ce rôle revient notamment aux moteurs d'inférence, qui assurent l'exécution du modèle en fonction des opérations et des ressources matérielles prises en charge par la plateforme.

1.4.2 Moteurs d'inférence

Dans ce contexte, le moteur d'inférence constitue la couche logicielle chargée d'adapter l'exécution du modèle aux caractéristiques de la plateforme cible. Son rôle est particulièrement important sur les systèmes *Edge* : il doit notamment permettre d'exploiter les représentations de faible précision présentées précédemment, de limiter l'empreinte mémoire et d'utiliser efficacement les unités de calcul disponibles.

Pour cela, les moteurs d'inférence intègrent différentes optimisations au niveau de l'exécution, telles que des *kernels* spécialisés, la fusion d'opérations, la planification et la réutilisation de la mémoire ou encore la délégation de certaines parties du graphe de calcul à des *backends* matériels spécifiques. La prise en charge de ces mécanismes dépend du moteur, du modèle et du matériel ciblé ; une même optimisation du modèle ne conduit donc pas nécessairement aux mêmes gains sur toutes les configurations.

Plusieurs moteurs ont été développés ou adaptés pour l'inférence locale des modèles de langage. `llama.cpp`, par exemple, repose sur une implémentation principalement en C/C++ et prend en charge de nombreux schémas de quantification ainsi que différents *backends* pour

CPU et accélérateurs, notamment CUDA, Vulkan et SYCL. Il permet également de répartir l'exécution d'un modèle entre CPU et GPU lorsque cela est nécessaire [11].

D'autres *runtimes* suivent une approche davantage orientée vers la délégation des calculs aux accélérateurs disponibles. ExecuTorch, destiné notamment aux plateformes mobiles et embarquées, permet par exemple de partitionner le graphe du modèle et d'en déléguer certaines parties à des *backends* optimisés pour CPU, GPU, *Neural Processing Unit* (NPU) ou *Digital Signal Processor* (DSP), tout en conservant une exécution sur CPU pour les opérations qui ne sont pas prises en charge par l'accélérateur ciblé [24]. Des environnements tels qu'ONNX Runtime ou MLC LLM proposent également différents mécanismes de compilation et d'accélération pour adapter l'exécution aux plateformes cibles.

Le moteur d'inférence détermine ainsi la manière dont les opérations du modèle sont effectivement mises en œuvre et réparties sur les ressources de calcul disponibles. Les possibilités d'optimisation restent cependant conditionnées par les caractéristiques du matériel sous-jacent. Il convient donc maintenant de considérer les principales architectures matérielles rencontrées dans les plateformes *Edge*.

1.4.3 Plateformes matérielles

Les plateformes *Edge* reposent de plus en plus sur des architectures hétérogènes associant un CPU à différents accélérateurs, tels que des GPUs, NPUs ou DSPs. Ces unités présentent des capacités de calcul, des niveaux de performance et des jeux d'opérations pris en charge différents. Les accélérateurs spécialisés, comme les NPUs, peuvent ainsi être particulièrement efficaces pour certaines opérations ou précisions numériques, mais leur utilisation dépend de la prise en charge effective des opérateurs du modèle. Les opérations non prises en charge doivent alors être exécutées sur une autre unité, ce qui peut introduire des transferts supplémentaires et limiter les gains attendus de l'accélération.

Ces différences concernent également les formats numériques pris en charge et l'organisation du sous-système mémoire, particulièrement importante pour l'inférence des modèles de langage. Les plateformes *Edge* présentent à ce niveau des architectures variées : le Raspberry Pi 5 repose principalement sur un CPU Arm Cortex-A76 associé à une mémoire *Dynamic Random-Access Memory* (DRAM) LPDDR4X, tandis que les plateformes NVIDIA Jetson combinent un CPU Arm, un GPU et d'autres accélérateurs autour d'une mémoire partagée. Le choix de la ressource d'exécution dépend ainsi de l'ensemble de ces caractéristiques matérielles et de leur prise en charge par le moteur d'inférence [43].

Ces différences matérielles influencent les performances de la plateforme, tout comme la configuration des ressources, qui peut être ajustée à travers différents mécanismes de gestion de la puissance et affecte à la fois les performances et l'énergie consommée.

1.5 Gestion et mesure de l'énergie

Puisque cette étude s'intéresse à l'énergie consommée pendant l'inférence, un enjeu important sur les systèmes contraints, cette section précise les grandeurs utilisées pour la caractériser, les mécanismes permettant d'agir sur la puissance d'une plateforme ainsi que les principales méthodes permettant de la mesurer.

1.5.1 Puissance, énergie et composantes de la consommation

La puissance électrique $P(t)$ représente le taux instantané auquel l'énergie est consommée et s'exprime en watts (W). L'énergie consommée pendant une durée comprise entre t_0 et t_1 correspond à l'intégrale de cette puissance :

$$E = \int_{t_0}^{t_1} P(t)\, dt, \tag{1.2}$$

où E est exprimée en joules (J). Lorsque seule la puissance moyenne P_{avg} est considérée sur une durée T, cette relation devient $E = P_{\text{avg}}T$.

À l'échelle d'un circuit *Complementary Metal-Oxide-Semiconductor* (CMOS), cette puissance consommée peut être décomposée, en première approximation, en une composante statique et une composante dynamique :

$$P_{\text{tot}} = P_{\text{stat}} + P_{\text{dyn}}. \tag{1.3}$$

La puissance statique provient principalement des courants de fuite présents tant que le circuit est alimenté et peut être approximée par $P_{\text{stat}} = VI_{\text{leak}}$. Elle dépend notamment de la technologie, de la tension d'alimentation et de la température. La puissance dynamique est principalement liée aux commutations des transistors lors de l'activité du circuit et peut être approximée par $P_{\text{dyn}} = \alpha C_{\text{eff}} V^2 f$ [23], où α représente le facteur d'activité, C_{eff} la capacité effective commutée, V la tension d'alimentation et f la fréquence de fonctionnement.

Un point à noter à l'échelle d'une plateforme complète est que la puissance mesurée au repos, souvent notée P_0 ou P_{idle}, ne doit pas être confondue avec la seule puissance statique du circuit. La puissance au repos inclut également l'activité résiduelle du système d'exploitation, de la mémoire, des régulateurs et des autres composants qui restent actifs en l'absence de la charge étudiée. Cette puissance de fond constitue une composante de l'énergie totale dès lors que la plateforme reste alimentée pendant l'exécution.

1.5.2 Gestion de la puissance

Pour agir sur les différentes composantes de la puissance, les plateformes de calcul intègrent plusieurs mécanismes de gestion énergétique. Ceux-ci permettent principalement d'adapter les points de fonctionnement des composants, de désactiver temporairement les ressources inutilisées ou encore de contraindre conjointement plusieurs paramètres à travers des modes de puissance.

- ***Dynamic Voltage and Frequency Scaling*** **(DVFS)** : Ce mécanisme consiste à faire fonctionner un composant à différents points de fonctionnement associant une fréquence à une tension d'alimentation. En pratique, le matériel expose un ensemble discret de points autorisés, parmi lesquels une politique de contrôle sélectionne celui qui est adapté à la charge ou aux contraintes imposées par le système.

  La diminution de la fréquence peut permettre de réduire la tension nécessaire au fonctionnement du circuit et donc sa puissance dynamique. Elle peut cependant augmenter le temps d'exécution. Inversement, une fréquence plus élevée augmente généralement

la puissance instantanée mais peut réduire la durée du traitement. Le point de fonctionnement qui minimise la puissance n'est donc pas nécessairement celui qui minimise l'énergie, et son intérêt dépend de la sensibilité de la charge à la fréquence [22].

Le DVFS peut être appliqué à plusieurs domaines matériels. Pour le CPU, les différents niveaux de performance sont couramment décrits par des états de performance, ou *P-states*, correspondant à différents points de fonctionnement en fréquence et en tension. Sous Linux, l'infrastructure `CPUFreq` permet de sélectionner dynamiquement la fréquence selon une politique définie par un *governor*. Pour le GPU, des mécanismes analogues permettent d'adapter sa fréquence à son niveau d'activité, notamment à travers l'infrastructure `devfreq` sur certaines plateformes. Le sous-système mémoire peut également disposer de sa propre mise à l'échelle dynamique : sur les plateformes Jetson Orin, la fréquence de l'*External Memory Controller* (EMC) et de la DRAM est adaptée aux demandes de bande passante et à l'utilisation effective de la mémoire [27].

Les fréquences sélectionnées par ces mécanismes restent néanmoins soumises aux limites thermiques et électriques de la plateforme. Avant d'atteindre des conditions susceptibles d'endommager le matériel, des mécanismes de *throttling* peuvent imposer une réduction des fréquences. Cette limitation peut être déclenchée par le logiciel de gestion thermique ou directement par le matériel lorsque certains seuils sont atteints [27].

- **Désactivation des ressources et états de faible consommation** : Une autre manière de réduire la consommation consiste à limiter l'activité des composants qui ne sont pas nécessaires. Le *clock gating* interrompt l'horloge d'un bloc matériel afin d'éviter les commutations inutiles, tandis que le *power gating* coupe son alimentation et permet ainsi de réduire également la consommation statique. Ces mécanismes introduisent toutefois un compromis avec le temps nécessaire pour réactiver la ressource.

  Sur CPU, ces principes apparaissent notamment à travers les états d'inactivité, ou *C-states*. Sous Linux, l'infrastructure `CPUIdle` sélectionne un état de faible consommation lorsqu'un cœur n'a plus de tâche à exécuter. Selon l'architecture, les états les plus profonds peuvent aller d'une simple attente ou désactivation de l'horloge jusqu'au *power gating* du cœur ou du cluster. Il est également possible de retirer certains cœurs de l'ensemble des processeurs actifs afin de réduire les ressources disponibles.

  Des mécanismes similaires existent pour les accélérateurs. Certains blocs GPU ou accélérateurs spécialisés peuvent être placés dans des états de faible consommation lorsqu'ils sont inutilisés, par exemple sur certaines plateformes Jetson, le nombre de *Texture Processing Clusters* (TPCs) disponibles sur le GPU peut être réduit [27].

- **Modes de puissance** : Les paramètres des mécanismes précédents peuvent enfin être combinés dans des configurations globales appelées *modes de puissance*. Un mode de puissance définit un ensemble de contraintes appliquées à plusieurs ressources de la plateforme, plutôt qu'un niveau de puissance instantanée.

  Sur les plateformes NVIDIA Jetson, l'outil `nvpmodel` permet ainsi de sélectionner des profils prédéfinis ou personnalisés qui peuvent imposer conjointement des limites sur les fréquences du CPU, du GPU et de l'EMC, ainsi que sur le nombre de cœurs CPU et de TPCs GPU [27]. Deux modes associés à des budgets de puissance différents peuvent donc modifier simultanément plusieurs caractéristiques matérielles, et un même budget

peut être obtenu à travers différentes répartitions des ressources.

1.5.3 Mesure de la puissance

La caractérisation énergétique nécessite d'observer la puissance consommée pendant une fenêtre d'exécution puis d'en déduire l'énergie. Plusieurs méthodes peuvent être utilisées, qui se distinguent notamment par leur principe de mesure, leur frontière d'observation, leur résolution temporelle et leur précision.

- **Estimation logicielle** : La puissance est estimée à partir d'informations accessibles au logiciel, telles que l'utilisation des unités de calcul, leurs fréquences, les compteurs de performance ou l'activité mémoire. Ces informations alimentent des modèles analytiques ou empiriques qui associent l'état observé de la plateforme à une consommation estimée. La frontière d'observation peut être définie à différents niveaux, par exemple celui d'un processus ou d'un composant, selon les informations disponibles et le modèle utilisé, avec une précision plus ou moins élevée. Des outils comme PowerAPI, à travers son modèle SmartWatts, utilisent par exemple des compteurs de performance et d'autres métriques système pour estimer la puissance consommée [10] au niveau des processus.

- **Mesures matérielles**
  - **Mesures internes à la plateforme** : Certaines architectures exposent des informations énergétiques à travers des compteurs ou des circuits intégrés à la plateforme. Sur les processeurs Intel, *Running Average Power Limit* (RAPL) fournit des compteurs d'énergie pour différents domaines du processeur. Ces valeurs sont maintenues par la plateforme et accessibles par logiciel à travers des registres dédiés ; leur calcul peut combiner des mesures électriques et des estimations pour certains domaines [15].

    Sur les plateformes NVIDIA Jetson, des circuits INA3221 mesurent directement la tension et le courant sur plusieurs rails d'alimentation. Ces valeurs sont accessibles sous Linux à travers l'interface `hwmon` ou à travers des outils de plus haut niveau comme Alumet, permettant ainsi d'avoir la puissance des domaines correspondants [27].

    Ces mécanismes facilitent l'automatisation des mesures, mais leur interprétation dépend de la frontière couverte par chaque compteur ou rail ainsi que de leur fréquence de mise à jour maximale.
  - **Mesures externes** : Une mesure indépendante peut être obtenue en plaçant un instrument entre l'alimentation et la plateforme étudiée. Par exemple, un analyseur de puissance mesure la tension et le courant fournis au système et permet d'en déduire la puissance selon $P(t) = V(t)I(t)$. Cette approche permet de mesurer une frontière plus large, par exemple l'ensemble de la carte, et constitue une référence utile pour comparer ou valider les informations fournies par la plateforme [2]. Elle ne permet cependant généralement pas de distinguer directement la consommation des différents composants internes.

Le choix de la méthode de mesure doit donc tenir compte des différents facteurs présentés ci-dessus, puisque les outils et les protocoles employés peuvent conduire à des écarts dans les valeurs obtenues qui doivent être considérés lors de l'interprétation et de la comparaison des résultats [2].

1.6 Modélisation de l'énergie et des performances

L'analyse de la consommation énergétique et des performances des applications a donné lieu à différentes catégories de modèles. On distingue notamment les modèles analytiques, qui représentent le comportement du système ou de l'application à partir d'équations et d'hypothèses simplificatrices, tels que les modèles de files d'attente, le modèle ECM (*Execution-Cache-Memory*) ou encore le modèle Roofline. On trouve également des modèles empiriques, construits à partir de mesures expérimentales et s'appuyant sur des méthodes statistiques ou d'apprentissage à partir de métriques mesurées, ainsi que des modèles par simulation, qui reproduisent le fonctionnement du système à un niveau d'abstraction donné.

Parmi ces catégories, les modèles analytiques nous intéressent particulièrement puisqu'ils permettent d'interpréter explicitement l'influence des différentes grandeurs considérées. En particulier, le modèle Roofline est bien adapté à notre étude en raison de sa simplicité et de son lien direct avec les ressources système, notamment les capacités de calcul et de mémoire de la plateforme. Le Roofline met en relation l'intensité arithmétique d'une charge avec les plafonds de calcul et de bande passante, permettant ainsi de visualiser l'effet de ces ressources sur les performances et, à travers son extension énergétique, sur l'efficacité énergétique.

Ces deux représentations sont présentées ci-dessous et seront utilisées dans la suite de ce mémoire pour caractériser les capacités CPU et GPU et y positionner les charges d'inférence étudiées.

1.6.1 *Time Roofline*

Une charge réalisant une quantité de calcul W, exprimée en FLOP, pour un volume de données Q, exprimé en octets, est caractérisée par son intensité arithmétique I :

$$I = \frac{W}{Q}. \tag{1.4}$$

Pour une ressource et une configuration données, le modèle considère un plafond de calcul F_{roof}, exprimé en FLOP/s, et un plafond de bande passante mémoire BW_{roof}, exprimé en octets/s. La performance associée à une intensité arithmétique I est donnée par :

$$P_{\text{roof}}(I) = \min\left(F_{\text{roof}}, BW_{\text{roof}} I\right). \tag{1.5}$$

Les deux limites se croisent pour l'intensité :

$$I_T = \frac{F_{\text{roof}}}{BW_{\text{roof}}}, \tag{1.6}$$

qui définit le point de basculement temporel. En dessous de I_T, la performance est limitée par la bande passante mémoire et la charge se situe dans la région *memory-bound*. Au-dessus

de cette valeur, elle est limitée par le plafond de calcul et se situe dans la région *compute-bound*.

Cette classification dépend à la fois de l'intensité arithmétique de la charge et des plafonds associés à la ressource matérielle considérée. Une même charge peut donc se situer dans des régions différentes selon la plateforme, la ressource d'exécution ou sa configuration.

1.6.2 *Energy Roofline*

L'*Energy Roofline* étend la représentation précédente à l'efficacité énergétique. Pour une exécution réalisant une quantité de calcul W et manipulant un volume de données Q pendant une durée t, l'énergie totale peut être modélisée par :

$$E = P_0 t + \epsilon_f W + \epsilon_m Q, \tag{1.7}$$

où P_0 représente la puissance de fond de la plateforme, ϵ_f le coût énergétique par opération de calcul et ϵ_m le coût énergétique par octet de données manipulé.

En utilisant l'intensité arithmétique $I = W/Q$, soit $Q = W/I$, puis en divisant l'Équation 1.7 par W, l'énergie consommée par FLOP s'écrit :

$$\frac{E}{W} = \epsilon_f + \frac{\epsilon_m}{I} + P_0 \frac{t}{W}. \tag{1.8}$$

La performance étant définie par $P = W/t$, le terme t/W correspond à l'inverse de la performance. En considérant la performance limite $P_{\text{roof}}(I)$ définie par le *Time Roofline*, l'énergie par FLOP associée au modèle devient :

$$e_{\text{roof}}(I) = \epsilon_f + \frac{\epsilon_m}{I} + \frac{P_0}{P_{\text{roof}}(I)}. \tag{1.9}$$

En tenant compte à la fois des coefficients énergétiques et de la contribution de la puissance de fond au temps d'exécution, le point de basculement énergétique est défini par Prashanthi et al. [29] comme :

$$I_E = \frac{\epsilon_m + \dfrac{P_0}{BW_{\text{roof}}}}{\epsilon_f + \dfrac{P_0}{F_{\text{roof}}}}. \tag{1.10}$$

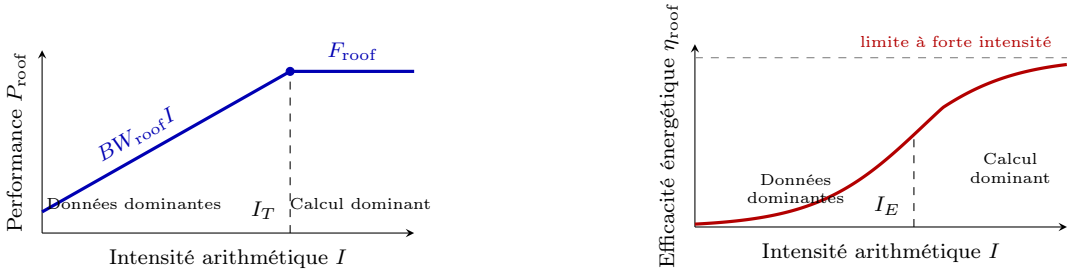
Ce point correspond à l'intensité arithmétique pour laquelle les contributions énergétiques effectives associées au calcul et aux transferts de données s'équilibrent. Pour $I < I_E$, la contribution liée aux données domine, tandis que pour $I > I_E$, celle liée au calcul devient dominante. Contrairement au point de basculement temporel, I_E dépend donc également de ϵ_f, de ϵ_m et de la puissance de fond P_0.

Dans la suite de ce mémoire, l'*Energy Roofline* est représenté en termes d'efficacité énergétique, soit l'inverse de l'énergie par FLOP :

$$\eta_{\text{roof}}(I) = \frac{1}{e_{\text{roof}}(I)}, \tag{1.11}$$

où η_{roof} est exprimée en FLOP/J. Cette représentation permet ainsi de mettre en relation l'intensité arithmétique d'une charge et l'efficacité énergétique associée aux limites du modèle.

La Figure 1.2 illustre les deux représentations. Le *Time Roofline* distingue les régions *memory-bound* et *compute-bound* autour de I_T, tandis que l'*Energy Roofline* distingue les régions où les contributions liées aux données ou au calcul dominent autour de I_E.



(a) *Time Roofline.* (b) *Energy Roofline.*

FIGURE 1.2 – Représentation schématique du *Time Roofline* et de l'*Energy Roofline*.

1.7 Conclusion

Ce chapitre a présenté les principaux concepts nécessaires à l'étude de l'inférence des modèles de langage sur des plateformes *Edge*, depuis l'architecture Transformer et le déroulement des phases de *prefill* et de *decode* jusqu'aux contraintes matérielles et logicielles de leur exécution. Les mécanismes de gestion de la puissance et les différentes méthodes de mesure énergétique ont également été introduits. Enfin, les modèles *Time Roofline* et *Energy Roofline* ont été présentés afin de disposer du cadre d'analyse qui sera utilisé dans la suite du mémoire pour mettre en relation les caractéristiques des charges étudiées avec les capacités de calcul, de mémoire et d'efficacité énergétique de la plateforme. Ces éléments constituent la base nécessaire pour analyser les travaux existants consacrés à la caractérisation et à l'optimisation énergétique de l'inférence.

Chapitre 2

État de l'art

2.1 Introduction

L'objectif de ce travail est de caractériser la consommation énergétique de l'inférence des LLMs sur des plateformes *Edge* afin d'identifier des leviers d'optimisation. Il convient donc d'examiner comment les travaux existants caractérisent et modélisent cette consommation, puis comment les facteurs étudiés sont exploités pour réduire l'énergie consommée.

Ce chapitre présente d'abord les principales approches de caractérisation et de modélisation énergétique de l'inférence, puis les stratégies d'optimisation agissant au niveau du modèle, de son exécution ou de la configuration de la plateforme. Une synthèse permet enfin de dégager les axes retenus pour le positionnement du travail.

2.2 Caractérisation et modélisation énergétique de l'inférence des LLMs

Les travaux consacrés à la caractérisation énergétique peuvent être distingués selon deux dimensions complémentaires. La première consiste à observer expérimentalement comment la consommation varie sous l'effet des différents facteurs de l'inférence, selon la granularité à laquelle l'exécution est considérée, et à en analyser les comportements. La seconde consiste à construire des modèles à partir de ces observations ou des caractéristiques de la charge et de la plateforme, puis à les utiliser soit pour estimer la consommation, soit pour relier les variations observées à ces caractéristiques.

2.2.1 Caractérisation des facteurs d'inférence

Les travaux de caractérisation cherchent d'abord à identifier les facteurs qui font varier l'énergie consommée. Ceux-ci peuvent être regroupés en facteurs applicatifs, tels que le modèle, la charge de travail ou le moteur d'inférence, en facteurs système, tels que les modes de puissance, les fréquences ou la quantité de ressources actives, et en facteurs matériels, notamment la ressource CPU ou GPU utilisée pour l'exécution. Une autre dimension concerne la granularité à laquelle ces effets sont observés, l'inférence pouvant être considérée globalement, décomposée en phases ou analysée à des niveaux plus fins.

Facteurs applicatifs, système et matériels. Sur les infrastructures GPU de serveurs, Samsi et al. [30] étudient l'effet du modèle, notamment à travers sa taille, ainsi que celui du matériel, à travers le type et le nombre de GPU mobilisés. Leur analyse met notamment en évidence qu'une augmentation des ressources peut réduire le temps d'exécution tout en augmentant la puissance consommée. L'énergie doit donc être étudiée à partir de ces deux dimensions.

Fernandez et al. [9] élargissent cette analyse en considérant conjointement la charge de travail, à travers les longueurs d'entrée et de sortie et le *batch*, le moteur d'inférence et la stratégie de décodage, ainsi que le matériel à travers différents GPU. Leurs résultats montrent notamment que l'effet énergétique d'une optimisation peut varier selon la charge, l'environnement logiciel et le matériel considérés, ce qui souligne l'intérêt de caractériser ces différents facteurs plutôt que d'évaluer une configuration unique.

Sur *Edge*, cette question inclut également la configuration des ressources de la plateforme. Arya et Simmhan [4] font varier, sur Jetson AGX Orin, la taille et la quantification du modèle, le *batch*, la longueur de séquence ainsi que des paramètres système à travers neuf modes de puissance qui agissent sur le CPU, le GPU et la mémoire. Ils caractérisent ainsi des configurations globales sans isoler systématiquement la contribution de chaque paramètre, mais montrent que la fréquence mémoire est particulièrement sensible : sa diminution entraîne une forte augmentation de la latence et de l'énergie dans les configurations étudiées. Husom et al. [14] se concentrent davantage sur la quantification et montrent, sur Raspberry Pi 4, que son effet sur l'énergie et le temps varie selon le modèle et le format numérique considéré, sans amélioration systématique avec les niveaux de quantification les plus faibles.
La ressource matérielle constitue elle-même un facteur de l'inférence. Elle peut correspondre à différents types de GPU, mais également au choix entre le CPU, le GPU ou d'autres accélérateurs matériels. Islam et al. [17] comparent plusieurs SLMs sur Raspberry Pi 5, Jetson Nano et Jetson Orin Nano, avec des exécutions CPU et GPU. L'exécution GPU y présente la meilleure efficacité énergétique parmi les configurations étudiées, et une tendance similaire est observée par Giedra et al. [12] sur Jetson Orin Nano Super : malgré une puissance instantanée plus élevée, l'accélération obtenue sur GPU réduit suffisamment le temps d'exécution pour améliorer l'efficacité énergétique par rapport au CPU.

Granularité de la caractérisation. Les facteurs précédents ne produisent pas nécessairement le même effet pendant toute l'inférence. Patel et al. [28] montrent avec Splitwise, sur des infrastructures GPU de serveurs, que le *prefill* sollicite davantage les capacités de calcul, tandis que le *decode* est davantage contraint par les accès mémoire, ce qui peut conduire à des comportements énergétiques différents entre les deux phases. Niu et al. [26] étudient précisément cette distinction du point de vue énergétique avec TokenPowerBench en séparant la consommation du *prefill* et du *decode*. Une mesure agrégée sur l'ensemble de la requête peut alors masquer des comportements énergétiques différents entre ces deux phases.

Cette séparation a également été étudiée sur *Edge*. Abstreiter et al. [1] distinguent le traitement du *prompt* de la génération dans leur évaluation de plusieurs modèles sur des plateformes CPU et GPU. Leurs résultats montrent notamment que le GPU est plus avantageux que le CPU pour les deux phases. La caractérisation par phase apporte donc une information qui disparaît lorsque l'inférence est étudiée uniquement dans son ensemble.

2.2.2 Modélisation de la consommation énergétique

Au-delà de la caractérisation expérimentale par facteur, certains travaux construisent des modèles afin de représenter la relation entre la consommation énergétique et les caractéristiques de l'inférence. Deux objectifs principaux apparaissent. Certains modèles visent avant tout à estimer la consommation de configurations qui n'ont pas été directement mesurées, tandis que d'autres cherchent à structurer les observations afin de relier les variations obtenues aux caractéristiques de la charge ou de la plateforme et à guider les choix.

Modélisation prédictive. Une première approche consiste à construire, à partir de mesures expérimentales, un modèle capable d'estimer l'énergie de configurations qui n'ont pas été directement évaluées. Cette approche permet de parcourir plus rapidement un espace de configurations trop important pour être exploré exhaustivement et peut ainsi aider à guider les décisions d'optimisation.

Sur *Edge*, Lahmer et al. [21] construisent un modèle empirique permettant d'estimer l'énergie de réseaux de neurones exécutés sur Jetson TX2 et Xavier NX à partir notamment de leur charge de calcul et des paramètres des couches convolutionnelles et entièrement connectées. Pour les LLMs, Wilkins et al. [39] utilisent principalement les nombres de *tokens* d'entrée et de sortie afin d'estimer le temps et l'énergie sur un système CPU–GPU, puis exploitent ces estimations pour orienter l'affectation des requêtes.

Des travaux comme ceux de Caravaca et al. [7] et WattGPU, proposé par Argerich et al. [3], cherchent davantage à généraliser la prédiction à des modèles ou à des GPU absents des mesures initiales. Ces approches sont adaptées à l'exploration de grands espaces, mais leur objectif principal reste l'estimation des grandeurs recherchées sans chercher à expliquer les mécanismes à l'origine des variations observées.

Modélisation pour interpréter les variations observées. L'identification expérimentale des facteurs et de leurs effets indique quelles configurations diffèrent, mais ne suffit pas toujours à relier ces différences aux besoins de la charge exécutée ou aux capacités de la plateforme. Certaines approches introduisent pour cela une représentation explicite de ces caractéristiques.

Prashanthi et al. [29] adoptent cette démarche avec Pagoda sur Jetson AGX Orin pour des réseaux de neurones profonds. Leur analyse repose sur l'intensité arithmétique, avec des modèles *Time Roofline* et *Energy Roofline* qui mettent respectivement cette intensité en relation avec les capacités de calcul et de bande passante, puis avec les coûts énergétiques associés au calcul, aux transferts mémoire et à la puissance de fond. Les variations entre configurations sont ensuite interprétées selon la dépendance de la charge au calcul ou à la mémoire.

Pour les LLMs, Vartziotis et al. [36] proposent un modèle analytique qui distingue le *prefill* du *decode* et décompose l'énergie du GPU entre le calcul, l'accès aux paramètres du modèle, les écritures dans le *KV cache* et les lectures associées à l'attention. Leur modèle prend en compte la taille du modèle, la longueur du contexte et le nombre de *tokens* générés. Il est cependant calibré sur des GPU NVIDIA H100 et ne considère pas la configuration des différentes ressources d'une plateforme *Edge* hétérogène.

2.2.3 Fiabilité de la mesure énergétique

Qu'il s'agisse de caractériser directement les variations de consommation ou de construire et de valider un modèle à partir de mesures, les données utilisées doivent être suffisamment fiables pour que les conclusions qui en sont tirées soient robustes. Jay et al. [19] comparent plusieurs outils logiciels à des instruments physiques et observent des écarts selon la méthode employée.

Cette question concerne directement les plateformes *Edge*. Shalavi et al. [34] montrent, sur plusieurs plateformes NVIDIA Jetson, que les capteurs internes peuvent sous-estimer la puissance mesurée extérieurement, avec des écarts atteignant 50 % dans certaines configurations. La confrontation avec une instrumentation externe devient alors utile pour quantifier l'écart de mesure et vérifier si celui-ci modifie les conclusions de la caractérisation ou de la modélisation.

2.3 Optimisation énergétique de l'inférence des LLMs

Les facteurs identifiés lors de la caractérisation peuvent ensuite être exploités comme leviers d'optimisation. Les approches existantes agissent sur un ou plusieurs de ces facteurs, certaines les considérant individuellement et d'autres de manière conjointe.

2.3.1 Optimisations applicatives et logicielles

Les optimisations applicatives et logicielles agissent sur le modèle ou sur son exécution sans modifier directement le point de fonctionnement du matériel. Les techniques de compression présentées au chapitre précédent réduisent notamment la taille, la précision numérique ou certaines structures du modèle [13, 14]. Leur effet énergétique dépend toutefois de la capacité du moteur d'inférence et du matériel à exploiter la transformation appliquée.

Sarkar et al. [31] montrent, sur plusieurs modèles Transformer et plateformes embarquées, qu'une réduction de la taille du modèle par *pruning* ne s'accompagne pas nécessairement d'une réduction proportionnelle de l'énergie. Le coût réel de l'inférence reste donc lié à la manière dont le modèle compressé est exécuté sur la plateforme cible.

D'autres optimisations agissent davantage sur l'organisation ou l'implémentation de l'exécution. La distinction entre *prefill* et *decode* peut, par exemple, guider le placement des calculs. Splitwise, proposé par Patel et al. [28], affecte ces deux phases à des groupes de GPU distincts en fonction de leurs besoins respectifs. Cette approche, destinée aux infrastructures de serveurs, exploite directement les différences de comportement mises en évidence lors de la caractérisation.

L'implémentation du moteur constitue un autre levier. Jareño et al. [18] optimisent `llama.cpp` par une recherche automatique de séquences de passes LLVM visant à réduire le temps d'exécution et l'énergie sur Apple M4.

2.3.2 Optimisations du système

Les optimisations système agissent principalement sur les fréquences, les limites de puissance ou la quantité de ressources actives afin d'adapter le point de fonctionnement à la

charge exécutée.

Chen et al. [8] proposent DVFSLM sur Jetson AGX Orin et Orin NX. Le *prefill* est exécuté avec des fréquences élevées, tandis que les fréquences du GPU et de l'EMC sont adaptées au cours du *decode* à partir de modèles de puissance et de latence. La configuration système peut ainsi évoluer selon la phase de l'inférence.

EnerInfer, proposé par Zou et al. [44], sélectionne les fréquences du NPU et de la mémoire pour l'inférence locale en tenant compte du débit, de la puissance et de la température. Ces approches ne recherchent donc pas systématiquement une fréquence minimale ou maximale ; elles adaptent le point de fonctionnement aux contraintes de l'exécution.

2.3.3 Optimisations conjointes

Les approches conjointes agissent simultanément sur plusieurs niveaux de l'inférence lorsque les paramètres concernés interagissent.

Camel, proposé par Xu et al. [40], combine sur Jetson AGX Orin la taille du *batch* et la fréquence GPU afin de rechercher un compromis entre énergie et latence. PELM, proposé par Yang et Xia [41], associe le DVFS au *speculative decoding*, qui génère plusieurs *tokens* candidats avant leur vérification, et adapte également la profondeur de cette vérification. La charge de calcul et le point de fonctionnement matériel sont alors contrôlés conjointement.

À l'échelle des infrastructures de serveurs, DynamoLLM, proposé par Stojkovic et al. [35], adapte le nombre d'instances, la répartition du modèle sur plusieurs GPU et leur fréquence selon la charge et les contraintes de service. Ces approches mettent en évidence l'intérêt de considérer les interactions entre leviers plutôt que de rechercher indépendamment la meilleure valeur de chaque paramètre.

2.4 Synthèse et positionnement du travail

Les travaux précédents montrent que plusieurs leviers peuvent être exploités pour réduire l'énergie de l'inférence, au niveau du modèle, du logiciel ou de la configuration du système. En revanche, du point de vue matériel, les approches d'optimisation recensées portent le plus souvent sur une ressource d'exécution donnée, majoritairement le GPU seul, ou sur des infrastructures composées de plusieurs GPU, ou plus rarement sur le CPU seul. Les travaux qui comparent directement l'efficacité des exécutions CPU et GPU sont, quant à eux, davantage orientés vers la caractérisation, sans proposer de modélisation permettant d'interpréter ces différences.

Le CPU peut pourtant constituer une ressource d'exécution pertinente dans plusieurs situations. Il peut notamment représenter la principale ressource d'exécution sur certains dispositifs dépourvus d'accélérateur adapté. Sur les plateformes disposant à la fois d'un CPU et d'un GPU, il peut également être utilisé lorsque le GPU est mobilisé par d'autres applications. Il devient alors possible d'envisager le déport total ou partiel de certaines tâches LLM vers le CPU en fonction des contraintes de la plateforme, par exemple pour respecter un budget énergétique ou libérer le GPU pour une autre charge. De tels choix nécessitent néanmoins de caractériser les deux ressources afin de déterminer comment leurs performances et leur efficacité énergétique évoluent selon la charge exécutée.

Cette caractérisation doit également tenir compte des différentes dimensions mises en évidence dans l'état de l'art. Les facteurs applicatifs, notamment la charge de travail, sont largement étudiés. Le modèle et le moteur d'inférence constituent également des dimensions importantes à considérer pour évaluer la généralisation des résultats. Les facteurs système, tels que les modes de puissance, sont moins étudiés et sont souvent évalués à travers des configurations prédéfinies qui modifient simultanément plusieurs paramètres. Il devient alors difficile d'identifier l'effet propre de chacun d'eux.
La granularité de l'analyse constitue une autre dimension importante, puisqu'une caractérisation globale peut masquer les différences de comportement entre le *prefill* et le *decode*.

Notre problématique portant également sur la modélisation de la consommation, l'étude conjointe des paramètres système et des exécutions CPU et GPU conduit à s'appuyer sur une représentation permettant de relier les variations observées aux capacités de calcul et de mémoire de la plateforme, afin de mieux interpréter les différences entre les configurations.

Enfin, la méthode de mesure constitue un enjeu transversal. Les écarts entre les mesures internes de la plateforme et une instrumentation externe peuvent modifier les valeurs d'énergie obtenues. Leur comparaison permet donc de vérifier si ces écarts affectent également les variations relatives et les conclusions tirées sur les configurations étudiées.

Le Tableau 2.1 synthétise la couverture de ces dimensions dans les principaux travaux de caractérisation et de modélisation considérés.

Le présent travail se positionne ainsi à l'intersection de ces dimensions. Il compare les exécutions CPU et GPU en distinguant le *prefill* du *decode*, tout en faisant varier individuellement plusieurs paramètres système à travers les modes de puissance de la plateforme, au-delà des approches centrées sur le DVFS. Une modélisation des capacités de calcul et de mémoire est utilisée afin d'interpréter les variations observées en fonction des caractéristiques des charges de travail. Enfin, les mesures internes de la plateforme sont confrontées à celles d'un analyseur de puissance externe afin d'évaluer dans quelle mesure la méthode de mesure influence les conclusions obtenues. Pour le modèle et le moteur d'inférence, ils sont maintenus fixes dans cette étude, leur variation pouvant constituer une extension pour évaluer la généralisation des résultats à d'autres configurations applicatives.

2.5 Conclusion

L'état de l'art montre que la consommation énergétique de l'inférence dépend de facteurs applicatifs, système et matériels dont les effets peuvent varier selon la ressource et la phase considérées. Les travaux existants couvrent plusieurs de ces dimensions, mais leur combinaison avec une variation isolée des paramètres système, une modélisation explicative et une confrontation des mesures internes et externes reste peu étudiée sur *Edge*. Le chapitre suivant présente la méthodologie expérimentale retenue pour mener cette caractérisation.

TABLE 2.1 – Positionnement des travaux selon les dimensions étudiées.

Travail	**Plateforme**	**Facteurs applicatifs**		**Facteurs système**			**Matériel**	**Granularité**		**Modélisation**	**Mesures**
	Edge	Modèle	Charge	Moteur	Modes	DVFS	CPU+GPU	Globale	Phase	Modèle énergétique	Interne+externe
Samsi et al. [30]	×	✓	×	×	×	×	×	✓	×	×	×
Fernandez et al. [9]	×	✓	✓	✓	×	×	×	✓	×	×	×
Arya et Simmhan [4]	✓	✓	✓	×	✓	×	×	✓	×	×	×
Husom et al. [14]	✓	✓	×	×	×	×	×	✓	×	×	×
Islam et al. [17]	✓	✓	×	×	×	×	✓	✓	×	×	×
Giedra et al. [12]	✓	✓	×	×	×	×	✓	✓	×	×	✓
Abstreiter et al. [1]	✓	✓	✓	×	×	×	✓	✓	✓	×	✓
TokenPowerBench [26]	×	✓	✓	✓	×	×	×	✓	✓	×	×
Lahmer et al. [21]	✓	✓	✓	×	×	×	✓	×	×	✓	×
Wilkins et al. [39]	×	✓	✓	×	×	×	×	✓	×	✓	×
Caravaca et al. [7]	×	✓	✓	×	×	×	×	✓	×	✓	×
WattGPU [3]	×	✓	×	×	×	×	×	✓	×	✓	×
Pagoda [29]	✓	✓	✓	×	✓	✓	×	✓	×	✓	×
Vartziotis et al. [36]	×	✓	✓	×	×	×	×	✓	✓	✓	×
Ce travail	✓	×	✓	×	✓	×	✓	✓	✓	✓	✓

✓ : dimension étudiée ; × : dimension non étudiée dans le périmètre considéré. *Charge* désigne notamment la variation des longueurs d'entrée et de sortie ou du *batch*. *Modes* correspond à l'étude de modes de puissance, tandis que *DVFS* désigne une adaptation des fréquences par DVFS. *CPU+GPU* indique une comparaison explicite des deux ressources. *Phase* correspond à la distinction entre *prefill* et *decode*. *Modèle énergétique* indique l'utilisation d'une modélisation de la consommation ou de l'efficacité énergétique. *Interne+externe* indique la confrontation des mesures internes de la plateforme à une instrumentation externe.

TABLE 2.2 – Positionnement des travaux sur l'inférence LLM.

Travaux	**Plateforme**	**Matériel**	**Facteurs système**			**Facteurs applicatifs**			**Modélisation**	**Mesures**
	Edge	CPU+GPU	Modes	DVFS	Ressources actives	Modèle	Charge	Moteur	Modèle énergétique	Interne+externe
Edge – comparaison CPU/GPU Islam et al., Giedra et al., Abstreiter et al.	✓	✓	×	×	×	✓	△	×	×	△
Edge – facteurs applicatifs et système Arya et Simmhan, Husom et al.	✓	×	△	×	×	✓	△	×	×	×
Cloud / serveur – modélisation énergétique Wilkins et al., Caravaca et al., WattGPU, Vartziotis et al.	×	×	×	×	×	✓	△	×	✓	×
Ce travail	✓	✓	✓	×	×	×	✓	×	✓	✓

✓ : dimension étudiée par tous les travaux du groupe ; △ : dimension étudiée par certains travaux du groupe ; × : dimension non étudiée.
Ressources actives désigne la variation explicite du nombre de ressources matérielles actives, par exemple les cœurs CPU ou les unités GPU.

Chapitre 3

Méthodologie de caractérisation

3.1 Introduction

À partir du positionnement établi au chapitre précédent, ce chapitre présente la démarche retenue pour caractériser les performances et la consommation énergétique de l'inférence des LLMs.

Les questions de recherche qui structurent l'étude sont d'abord formulées. La démarche mise en place pour y répondre est ensuite présentée, avant de détailler sa mise en œuvre expérimentale à travers la plateforme, les configurations système, l'instrumentation, la construction des *Rooflines* et la caractérisation des charges de travail LLM. Les résultats obtenus sont présentés et analysés dans le chapitre suivant.

3.2 Questions de recherche

À partir du positionnement établi dans l'état de l'art, cette étude vise à caractériser et à modéliser l'efficacité énergétique des exécutions CPU et GPU en fonction de la charge et des configurations système. La caractérisation distingue les phases de *prefill* et de *decode* et s'appuie sur des mesures énergétiques internes et externes afin d'évaluer également l'influence de la méthode de mesure sur les conclusions obtenues.

Pour relier les variations observées aux capacités de la plateforme, le modèle *Roofline* présenté dans la Section 1.6 est retenu. Il permet de mettre en relation l'intensité arithmétique des charges avec les capacités de calcul et de bande passante du CPU et du GPU, puis d'étudier leur évolution selon les configurations système. Et comme vu dans l'état de l'art l'énergie est liée au temps d'exécution, la démarche s'appuie donc conjointement sur les *Time Rooflines* et les *Energy Rooflines*.

À partir de ces choix, l'étude est structurée autour de trois questions de recherche :

- **Q1 :** Comment l'écart entre CPU et GPU en performance et en efficacité énergétique évolue-t-il avec l'intensité arithmétique et les configurations système ?
- **Q2 :** Dans quelle mesure les écarts CPU/GPU mis en évidence par les *Rooflines* se retrouvent-ils lors de l'exécution réelle des phases de *prefill* et de *decode*, et comment ces écarts évoluent-ils selon les configurations système ?

- **Q3 :** Les mesures énergétiques internes et externes conduisent-elles aux mêmes variations relatives et au même classement des configurations ?

3.3 Démarche de caractérisation

Pour répondre aux trois questions de recherche, la démarche est organisée en plusieurs étapes. Les *Rooflines* du CPU et du GPU sont d'abord construits séparément sous les différentes configurations système à partir de microbenchmarks. Leur comparaison permet d'étudier l'évolution des écarts de performance et d'efficacité énergétique entre les deux ressources et de répondre à Q1.

Les charges de travail LLM sont ensuite exécutées et représentées par leur quantité de calcul W et leur volume de données Q, dont est déduite l'intensité arithmétique. Leur positionnement sur les *Rooflines* permet de confronter les écarts observés sur les modèles aux comportements mesurés lors du *prefill* et du *decode*, afin de répondre à Q2.

Enfin, les mesures énergétiques internes et externes collectées lors de ces mêmes exécutions sont comparées en valeur absolue, en variation relative et en termes de classement des configurations afin de répondre à Q3.

3.3.1 Étape 1 : Modélisation *Roofline*

La première étape consiste à instancier séparément les modèles *Time Roofline* et *Energy Roofline*, pour le CPU et le GPU sous chaque configuration système.

Les paramètres des *Rooflines* sont déterminés expérimentalement à partir de microbenchmarks plutôt que déduits des seules spécifications matérielles. Pour le *Time Roofline*, le plafond de calcul F_{roof} et le plafond de bande passante BW_{roof} sont mesurés. Pour l'*Energy Roofline*, la puissance de fond P_0 ainsi que les coefficients énergétiques ϵ_f et ϵ_m sont estimés à partir des mesures expérimentales. La procédure utilisée pour obtenir ces grandeurs est détaillée dans la Section 3.4.2.

3.3.2 Étape 2 : Exécution et modélisation des charges LLM

Après avoir défini les *Rooflines* de la plateforme, leur application aux charges de travail LLM nécessite d'exécuter ces dernières et d'estimer les mêmes grandeurs W et Q que celles utilisées pour définir l'intensité arithmétique. Dans notre cas, cette estimation repose sur le graphe d'exécution de l'inférence plutôt que sur une approximation dérivée uniquement de l'architecture théorique du modèle.

Pour chaque charge de travail, le nombre de FLOP utiles et le volume logique de données manipulées sont déterminés à partir des opérations du graphe, puis utilisés pour calculer l'intensité arithmétique selon l'Équation 1.4. Le *prefill* et le *decode* sont caractérisés séparément afin de tenir compte de leurs graphes d'exécution différents. Les règles de comptage ainsi que la méthode d'extraction et d'analyse des graphes sont détaillées dans la Section 3.4.3.

3.3.3 Étape 3 : Comparaison des mesures énergétiques

Les deux méthodes de mesure sont utilisées simultanément sur les mêmes exécutions. La comparaison porte sur les valeurs énergétiques obtenues, les variations relatives entre configurations et leur classement énergétique. Un écart sur les valeurs absolues n'a en effet pas les mêmes conséquences qu'une modification des tendances ou de la configuration identifiée comme la plus efficace.

3.4 Mise en œuvre expérimentale

La mise en œuvre repose sur un environnement expérimental commun à la construction des *Rooflines* et à la caractérisation des charges de travail LLM. La plateforme, la configuration applicative de référence, les configurations système et le protocole de mesure sont d'abord présentés, avant de détailler séparément les deux campagnes expérimentales.

3.4.1 Environnement expérimental commun

Plateforme matérielle

Les expériences sont réalisées sur le CPU et le GPU d'une carte NVIDIA Jetson Orin Nano Super Developer Kit. Le CPU comporte six cœurs Arm Cortex-A78AE, tandis que le GPU est basé sur l'architecture NVIDIA Ampere et comporte jusqu'à quatre TPC. Le CPU et le GPU partagent la mémoire LPDDR5 de la plateforme, dont la fréquence est contrôlée par l'EMC. La plateforme fonctionne sous Jetson Linux 36.5 avec JetPack 6.2.2.

Configuration applicative de référence

Le modèle retenu pour notre étude est `Qwen2.5-0.5B-Instruct` au format GGUF avec des poids en FP16. Ce format numérique est maintenu fixe pour l'ensemble de l'étude, car sa modification peut changer les *kernels* exécutés, la quantité de calcul, le volume de données manipulées ainsi que les performances obtenues. Les microbenchmarks de calcul utilisés pour construire les *Rooflines* sont également exécutés en FP16 afin de conserver une cohérence avec le modèle étudié.

Les charges de travail LLM sont exécutées avec `llama.cpp` [11], version `b9114`, également utilisée pour extraire les graphes d'exécution nécessaires au calcul de W et Q. Les paramètres spécifiques aux charges de travail ainsi qu'aux exécutions CPU et GPU sont détaillés dans la Section 3.4.3.

Configurations système retenues

Les *Rooflines* et les charges de travail LLM sont exécutées sous différents modes de puissance définis à partir de cinq paramètres système : la fréquence CPU, le nombre de cœurs CPU actifs, la fréquence GPU, le nombre de TPC GPU actifs et la fréquence de l'EMC. Et comme l'espace de configurations est très large, l'étude adopte une approche *One Factor At a Time* (OFAT) à partir d'une configuration de référence, notée `BO_CENTRAL`. Pour chaque

configuration dérivée, un seul paramètre est modifié par rapport à `B0_CENTRAL`, tandis que les autres conservent leur valeur de référence.

Les dix configurations retenues sont présentées dans le Tableau 3.1. La fréquence de l'EMC constitue un cas particulier, puisque sa valeur dans `B0_CENTRAL` correspond déjà à la fréquence maximale considérée ; seule une diminution de cette fréquence est donc étudiée.

TABLE 3.1 – Configurations système utilisées pour l'étude OFAT.

Configuration	**CPU [MHz]**	**Cœurs**	**GPU [MHz]**	**TPC**	**EMC [MHz]**
B0_CENTRAL	1344	4	816	3	3199
CPUF_LOW	960	4	816	3	3199
CPUF_HIGH	1728	4	816	3	3199
GPUF_LOW	1344	4	612	3	3199
GPUF_HIGH	1344	4	1020	3	3199
EMCF_LOW	1344	4	816	3	2133
CPUC_LOW	1344	2	816	3	3199
CPUC_HIGH	1344	6	816	3	3199
GPUT_LOW	1344	4	816	2	3199
GPUT_HIGH	1344	4	816	4	3199

Afin de garantir que chaque configuration est effectivement appliquée pendant les acquisitions, celle-ci est activée avec `nvpmodel` [1], puis les fréquences CPU, GPU et EMC, le nombre de cœurs CPU en ligne et le nombre de TPC actifs sont vérifiés. Le ventilateur est maintenu à une consigne fixe, tandis que les fréquences effectives, les températures ainsi que les éventuels événements de surintensité et les alarmes de la plateforme sont suivis pendant les mesures.

Protocole expérimental commun

Les deux campagnes suivent un protocole commun afin de conserver les mêmes conditions de mesure. Pour chaque acquisition, le mode de puissance est d'abord appliqué et vérifié, puis une phase de *warm-up* est exécutée hors de la fenêtre de mesure. Les acquisitions des deux instruments sont ensuite démarrées avant l'exécution de la charge.

La mesure interne est réalisée avec Alumet à partir du rail `VDD_IN` exposé par la plateforme Jetson, avec une période d'échantillonnage de 0,1 s. En parallèle, l'analyseur Keysight N6705C mesure la puissance à l'entrée de la carte ; il est piloté depuis le programme d'expérimentation avec PyVISA [2], à travers des commandes SCPI [3].

Les deux instruments enregistrent ainsi la même exécution. À la fin de la charge, les acquisitions sont arrêtées et l'énergie est obtenue par intégration de la puissance sur la fenêtre de mesure. Chaque point expérimental est répété trois fois, tandis que les étapes propres à chaque campagne sont précisées dans les sections correspondantes.

1. `nvpmodel` est l'outil NVIDIA utilisé sur les plateformes Jetson pour sélectionner et appliquer les modes de puissance définissant notamment les ressources actives et leurs limites de fréquence.

2. PyVISA est une bibliothèque Python permettant de communiquer avec des instruments de mesure via l'architecture VISA (*Virtual Instrument Software Architecture*).

3. SCPI (*Standard Commands for Programmable Instruments*) est un standard de commandes textuelles utilisé pour contrôler et interroger des instruments de mesure programmables.

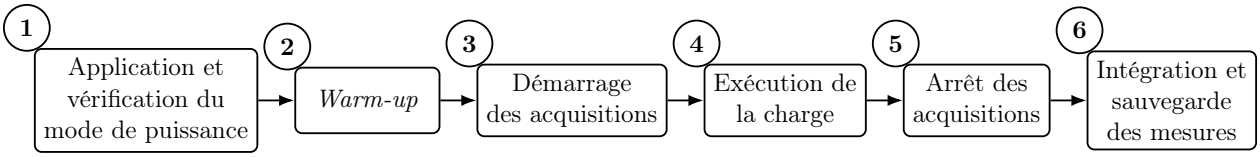


FIGURE 3.1 – Protocole commun d'une acquisition expérimentale.

3.4.2 Campagne 1 : Construction des *Rooflines*

Les *Rooflines* sont construits séparément pour le CPU et le GPU sous chacune des dix configurations système. Pour modéliser les capacités matérielles, on utilise en général des microbenchmarks indépendants de l'application conçus pour pousser les ressources jusqu'à leurs limites. Cependant, comme notre étude porte sur les LLMs, les microbenchmarks de calcul sont exécutés avec `llama.cpp` en FP16, en utilisant la bibliothèque `ggml`, afin de conserver un chemin de calcul aussi cohérent que possible avec la charge applicative.

Pour les deux *Rooflines*, les acquisitions suivent le protocole expérimental commun présenté précédemment. Pour chacune des configurations système définies dans le Tableau 3.1, une première exécution est faite hors de la fenêtre de mesure avant les acquisitions.

Construction du *Time Roofline*

Le plafond de calcul F_{roof} est obtenu à partir d'un GEMM FP16, tandis que le plafond de bande passante BW_{roof} est mesuré avec une opération Copy. Avant les acquisitions scientifiques, une phase de dimensionnement est réalisée pour ces deux microbenchmarks : la taille des matrices et le nombre d'éléments copiés sont augmentés progressivement par puissances de deux jusqu'à atteindre des performances stables. Les microbenchmarks sont ensuite exécutés avec les tailles correspondant à cette zone de stabilité afin d'obtenir les paramètres du *Time Roofline* défini par l'Équation 1.5. Les tailles d'entrée retenues sont conservées pour l'ensemble des configurations.

Construction de l'*Energy Roofline*

La construction de l'*Energy Roofline* nécessite d'estimer la puissance de fond P_0 ainsi que les coefficients ϵ_f et ϵ_m. Pour chaque configuration, P_0 est déterminé à partir de trois fenêtres indépendantes de 120 s.

Pour estimer les deux coefficients, trois familles de microbenchmarks présentant des intensités arithmétiques différentes sont utilisées : GEMM pour les charges dominées par le calcul, Copy pour les transferts mémoire et Triad pour des situations intermédiaires. Plusieurs tailles situées dans la zone de stabilité sont retenues dans chaque famille afin de disposer de plusieurs couples (W, Q) pour la régression.

Après soustraction de la contribution $P_0 t$, ϵ_f et ϵ_m sont estimés conjointement par une régression linéaire OLS sans constante, puisque la contribution de la puissance de fond a déjà été prise en compte :

$$\underset{\epsilon_f, \epsilon_m}{\operatorname{argmin}} \sum_i \left[(E_i - P_0 t_i) - \epsilon_f W_i - \epsilon_m Q_i \right]^2 . \tag{3.1}$$

3.4.3 Campagne 2 : Caractérisation des charges de travail LLM

À partir du modèle et de l'environnement logiciel définis précédemment, six charges de travail avec des tailles représentatives du cas *Edge* sont retenues afin de caractériser séparément les phases de *prefill* et de *decode* et d'étudier l'effet de leur longueur en fonction des paramètres système.

La campagne applicative suit le protocole commun présenté précédemment. Les exécutions sont répétées dans la fenêtre de mesure afin d'obtenir une durée d'acquisition d'environ 60 s pour chaque charge de travail. Et dans le cas particulier du *decode*, le contexte initial et le *KV cache* correspondant sont préparés hors de la fenêtre de mesure afin que celle-ci couvre uniquement la génération des nouveaux *tokens*.

Charges de travail et paramètres d'exécution

Pour les charges de travail LLM, le facteur applicatif étudié est la taille de la charge, à travers les longueurs d'entrée et de sortie, tandis que le modèle, le moteur d'inférence et le format numérique sont maintenus fixes. Trois longueurs sont considérées pour chaque phase. Le *prefill* est étudié avec des *prompts* de 128, 512 et 1024 *tokens*. Pour le *decode*, 128, 512 ou 1024 nouveaux *tokens* sont générés à partir d'un contexte initial de 128 *tokens*.

TABLE 3.2 – Charges de travail LLM utilisées pour la caractérisation.

Charge de travail	**Phase**	***Tokens* mesurés**	**KV initial**
P128	Prefill	128	–
P512	Prefill	512	–
P1024	Prefill	1024	–
O128	Decode	128	128
O512	Decode	512	128
O1024	Decode	1024	128

Les exécutions sont réalisées avec `llama-bench`. Pour le CPU, aucun bloc du modèle n'est déporté sur le GPU avec `n_gpu_layers=0`, tandis que `n_gpu_layers=99` est utilisé pour l'exécution GPU. Le nombre de *threads* CPU suit le nombre de cœurs actifs dans chaque configuration, soit deux, quatre ou six *threads*.

Le *batch* maximal est fixé à 1024 *tokens*, le *micro-batch* à 512 *tokens*, les caches K et V sont maintenus en FP16 et Flash Attention est désactivé afin de conserver un graphe logique comparable pour les exécutions CPU et GPU et d'utiliser une même définition de l'intensité arithmétique pour leur positionnement sur les *Rooflines*. Ces paramètres restent identiques pour l'ensemble des charges de travail et des configurations.

Extraction et quantification des graphes

Le positionnement des charges de travail sur les *Rooflines* nécessite de déterminer leur quantité de calcul W et leur volume logique de données Q. Ces grandeurs sont déterminées au préalable à partir d'une version instrumentée de `llama.cpp` permettant d'extraire le graphe construit lors de l'appel à `llama_decode()`, sur CPU et sur GPU.

Pour le *prefill*, le graphe correspondant à l'ensemble du *prompt* est extrait pour chacune des trois longueurs. Pour le *decode*, les graphes correspondant aux générations successives sont extraits à partir du contexte initial de 128 *tokens* puis agrégés sur la longueur considérée.

Pour chaque opération du graphe, W et Q sont déterminés à partir du type de l'opération ainsi que des dimensions et du format des tenseurs manipulés. Les tenseurs, c'est-à-dire les tableaux multidimensionnels qui transportent notamment les poids, les activations et les résultats intermédiaires entre les opérations, constituent ici les données consommées et produites par le graphe.

Le volume logique d'un tenseur T, noté $B(T)$, correspond à sa taille en octets et est calculé à partir de son nombre d'éléments et de son format numérique :

$$B(T) = N_{\text{elements}}(T) \times \text{sizeof}(\text{dtype}(T)). \tag{3.2}$$

Le volume Q d'une opération correspond alors à la somme des volumes logiques de ses tenseurs d'entrée et de sortie. Dans les expressions suivantes, src, src_0, src_1 et src_j désignent les tenseurs d'entrée selon l'opération considérée, tandis que dst désigne le tenseur de sortie. Lorsqu'un même tenseur est consommé par plusieurs opérations, sa lecture est comptée pour chacune d'elles.

Les règles de comptage retenues pour les différentes opérations observées dans les graphes sont présentées dans le Tableau 3.3. Les notations dépendent de l'opération considérée : N_{out} désigne le nombre d'éléments produits, K_{mm} la dimension commune de la multiplication matricielle, S le nombre de vecteurs normalisés et D leur dimension, N_{rot} le nombre d'éléments auxquels RoPE est appliqué, R le nombre de lignes du Softmax, K_{sm} leur longueur et N le nombre d'éléments traités par le GLU.

TABLE 3.3 – Règles de calcul de W, Q et de l'intensité arithmétique pour les opérations observées dans les graphes `llama.cpp`.

Opération	W **[FLOP]**	Q **[octets]**	$I = W/Q$
`MUL_MAT`	$2K_{\text{mm}}N_{\text{out}}$	$B(src_0) + B(src_1) + B(dst)$	$\frac{2K_{\text{mm}}N_{\text{out}}}{Q}$
`ADD`	N_{out}	$B(src_0) + B(src_1) + B(dst)$	$\frac{N_{\text{out}}}{Q}$
`MUL`	N_{out}	$B(src_0) + B(src_1) + B(dst)$	$\frac{N_{\text{out}}}{Q}$
`RMS_NORM`	$S(3D+3)$	$B(src) + B(dst)$	$\frac{S(3D+3)}{Q}$
`ROPE`	$3N_{\text{rot}}$	$\sum_j B(src_j) + B(dst)$	$\frac{3N_{\text{rot}}}{Q}$
`SOFT_MAX`	$5K_{\text{sm}}R$	$\sum_j B(src_j) + B(dst)$	$\frac{5K_{\text{sm}}R}{Q}$
`GLU`	$3N$	$\sum_j B(src_j) + B(dst)$	$\frac{3N}{Q}$
`GET_ROWS`	0	Lignes sélectionnées + indices + sortie	0
`SET_ROWS`	0	Source + indices + sous-ensemble écrit	0
`CONT`	0	$B(src) + B(dst)$	0
`VIEW`	0	0	–
`RESHAPE`	0	0	–
`PERMUTE`	0	0	–

Les valeurs de W et Q sont sommées sur l'ensemble des opérations du graphe pour obtenir les valeurs de la charge de travail complète. L'intensité arithmétique est ensuite calculée selon

l'Équation 1.4. Les résultats obtenus pour les six charges de travail sont présentés dans le Tableau 3.4.

TABLE 3.4 – Quantité de calcul, volume logique de données et intensité arithmétique des charges de travail LLM étudiées.

Charge de travail	W **[GFLOP]**	Q **[GB]**	I **[FLOP/B]**
P128	91.511	1.872	48.893
P512	376.645	5.231	72.008
P1024	775.980	11.603	66.880
O128	129.413	127.876	1.012
O512	529.148	513.831	1.030
O1024	1081.286	1032.317	1.047

Validation du comptage du graphe. Afin de valider le comptage analytique, une expérience complémentaire est réalisée sur un GPU NVIDIA T1000 8 GB avec NVIDIA Nsight Compute (NCU), qui permet d'accéder aux compteurs matériels nécessaires. La validation porte sur la charge de travail P512 en FP16.

TABLE 3.5 – Comparaison entre l'estimation issue du graphe et les mesures NCU pour P512.

Grandeur	**Estimation du graphe**	**Mesure NCU T1000**
W	376,645 GFLOP	378,407 GFLOP
Q	5,231 GB logiques	8,704 GB DRAM
I	72,008 FLOP/B logique	43,474 FLOP/B DRAM

Le nombre de FLOP mesuré par NCU ne diffère que de 0,47 % de l'estimation issue du graphe, ce qui confirme la cohérence du comptage de W pour cette charge de travail. En revanche, les valeurs de Q correspondent à deux grandeurs différentes : l'estimation du graphe représente le volume logique des tenseurs consommés et produits par les opérations, tandis que NCU mesure le trafic physique observé au niveau de la DRAM. Les intensités arithmétiques qui en résultent ne sont donc pas directement comparables et aucun facteur de correction du volume logique n'est déduit de cet écart.

3.5 Conclusion

Ce chapitre a présenté les questions de recherche ainsi que la démarche expérimentale mise en place pour y répondre. La méthodologie combine les *Time Rooflines* et les *Energy Rooflines*, l'étude contrôlée des configurations système, la caractérisation séparée du *prefill* et du *decode* ainsi que la comparaison de deux méthodes de mesure énergétique.

Le chapitre suivant s'appuie sur cette démarche pour analyser les résultats obtenus et répondre aux trois questions de recherche.

Chapitre 4

Résultats et discussion

4.1 Introduction

Ce chapitre présente et analyse les résultats obtenus à partir des campagnes expérimentales décrites au chapitre précédent. L'analyse est structurée selon les trois questions de recherche définies dans la méthodologie. L'écart entre CPU et GPU en fonction de l'intensité arithmétique et des configurations système est d'abord étudié afin de répondre à Q1. Les charges de travail de *prefill* et de *decode* sont ensuite positionnées sur les *Rooflines* et confrontées aux mesures applicatives pour répondre à Q2. Enfin, les mesures énergétiques internes et externes sont comparées afin de répondre à Q3.

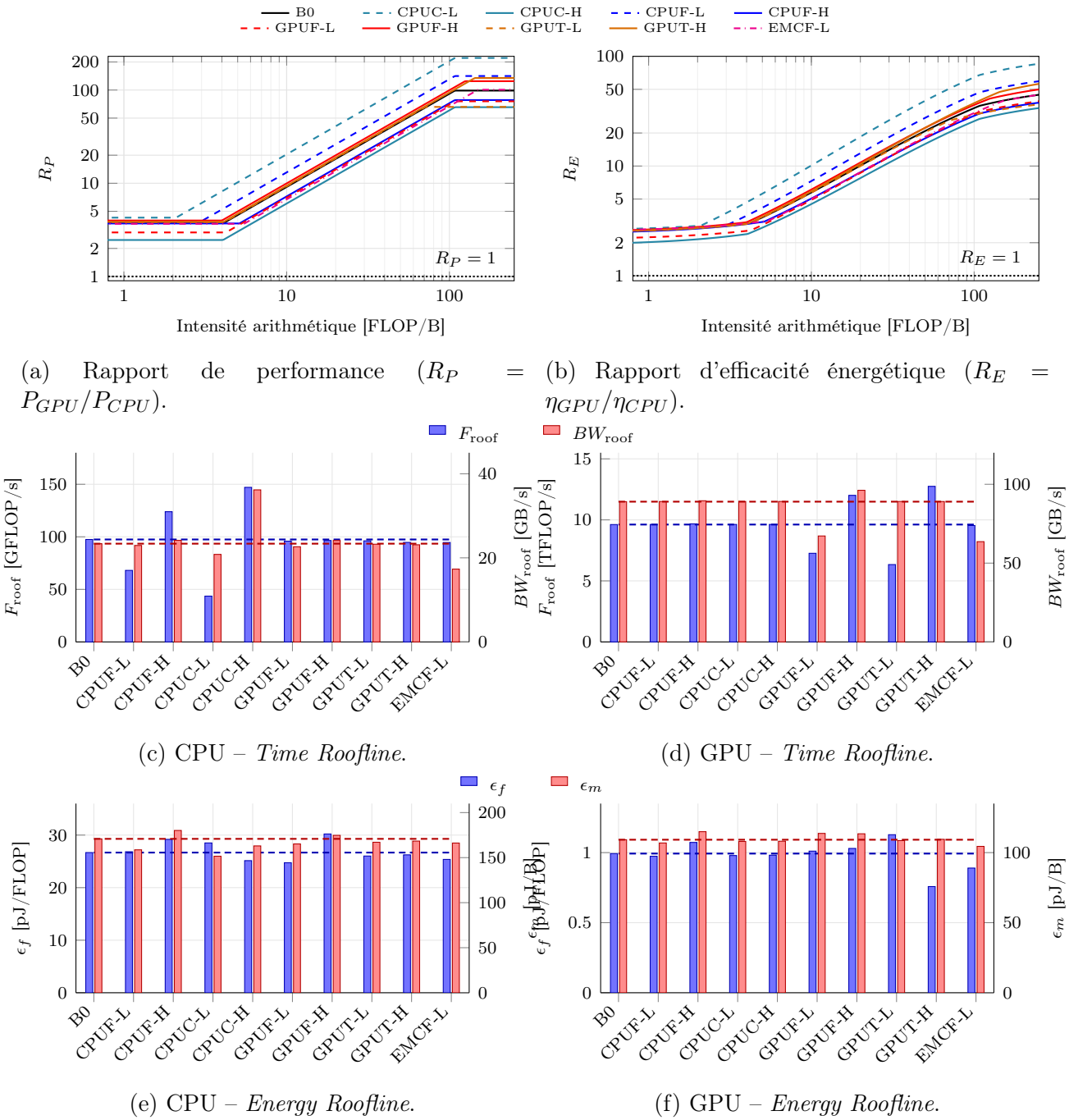
4.2 Q1 – Écart entre CPU et GPU selon l'intensité arithmétique et les configurations système

La première question de recherche vise à déterminer comment l'écart entre CPU et GPU en performance et en efficacité énergétique évolue avec l'intensité arithmétique et les paramètres système. Pour le quantifier, on considère les rapports :

$$R_P(I) = \frac{P_{\text{roof}}^{GPU}(I)}{P_{\text{roof}}^{CPU}(I)}, \qquad R_E(I) = \frac{\eta_{\text{roof}}^{GPU}(I)}{\eta_{\text{roof}}^{CPU}(I)}. \tag{4.1}$$

Une valeur proche de 1 correspond à des capacités proches entre les deux *backends*. Les Figures 4.1(a) et (b) représentent directement ces deux rapports pour l'ensemble des configurations OFAT étudiées. Les quatre panneaux suivants présentent, en valeurs absolues, les paramètres mesurés des *Time Rooflines* et des *Energy Rooflines* ; ils seront utilisés ensuite pour expliquer l'origine des variations observées dans R_P et R_E.

Les Figures 4.1(a) et (b) apportent une première réponse à Q1. Sur la plateforme étudiée, les plafonds GPU restent supérieurs aux plafonds CPU sur toute la plage d'intensité arithmétique considérée, aussi bien en performance qu'en efficacité énergétique. Leur écart n'est cependant pas constant : il est nettement plus faible à basse intensité arithmétique et augmente lorsque le calcul devient prépondérant. Les configurations système modifient ensuite l'amplitude de cet écart selon les ressources qu'elles affectent.



(a) Rapport de performance ($R_P = P_{GPU}/P_{CPU}$).

(b) Rapport d'efficacité énergétique ($R_E = \eta_{GPU}/\eta_{CPU}$).

(c) CPU – *Time Roofline*.

(d) GPU – *Time Roofline*.

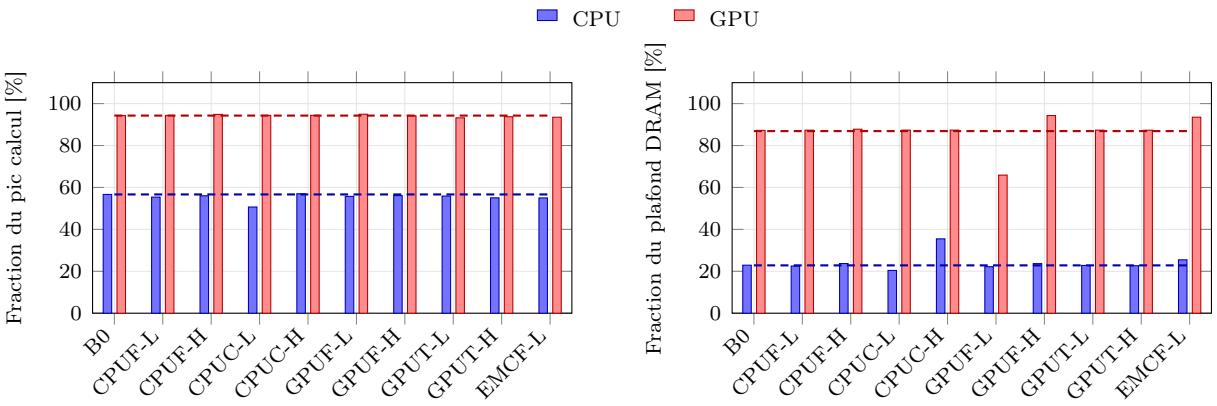
(e) CPU – *Energy Roofline*.

(f) GPU – *Energy Roofline*.

FIGURE 4.1 – Écarts GPU/CPU et paramètres des Rooflines selon les configurations OFAT. Les lignes pointillées des panneaux (c)–(f) indiquent les valeurs de référence B0.

Par exemple, dans la configuration B0, à faible intensité, le rapport de performance tend vers le rapport des bandes passantes, $\frac{BW_{\text{GPU}}}{BW_{\text{CPU}}} = \frac{88{,}97}{23{,}37} \simeq 3{,}81$. À forte intensité, il tend vers le rapport des plafonds de calcul, $\frac{F_{\text{GPU}}}{F_{\text{CPU}}} = \frac{9617}{97{,}48} \simeq 98{,}7$.
L'avantage du GPU est donc beaucoup moins marqué lorsque les transferts mémoire dominent que lorsque l'exécution est limitée par le calcul. Le même phénomène apparaît du point de vue énergétique : en B0, R_E vaut environ 2,55 à $I = 0{,}7$ FLOP/B, 20,9 à $I \simeq 50$ FLOP/B et 26,6 à $I \simeq 70$ FLOP/B.
Ce résultat suggère néanmoins qu'il peut être propre à la plateforme considérée. Zhang et Huang [42] montrent en effet que, sur un iPhone 15 Pro avec `llama.cpp`, une exécution sur CPU peut surpasser le GPU pour certains petits modèles, notamment Qwen2-0.5B et Llama-3.2-1B. Ils attribuent ce comportement à une meilleure exploitation du parallélisme CPU ainsi qu'aux surcoûts associés au chemin d'exécution GPU. Bien que leurs conditions expérimentales diffèrent des nôtres, cette comparaison montre que le rapport de performance entre CPU et GPU peut dépendre de l'architecture et ne peut donc pas être généralisé. Sur le Jetson Orin Nano Super étudié ici, le GPU conserve des capacités supérieures sur l'ensemble de la plage considérée, même si son avantage se réduit dans la région de faible intensité.

Le chemin d'exécution `llama.cpp/ggml` utilisé par nos microbenchmarks constitue un second facteur à prendre en compte, puisqu'il n'exploite pas la même fraction du potentiel matériel sur les deux *backends*. La Figure 4.2 met ainsi en regard les plafonds empiriques mesurés et les capacités théoriques associées à chaque configuration.



(a) Capacité de calcul.

(b) Bande passante mémoire.

FIGURE 4.2 – Fraction des capacités théoriques atteinte par les plafonds empiriques. Les lignes pointillées indiquent les valeurs de référence B0.

La fraction indiquée dans cette figure correspond au rapport entre le plafond empirique obtenu par les microbenchmarks et le plafond théorique associé à la configuration :

$$U_F = \frac{F_{\text{roof}}}{F_{\text{th}}}, \qquad U_{BW} = \frac{BW_{\text{roof}}}{BW_{\text{DRAM,th}}}. \tag{4.2}$$

Ces rapports ne mesurent pas uniquement l'efficacité du logiciel : ils quantifient la fraction du potentiel matériel effectivement atteinte avec le chemin d'exécution utilisé.
Une différence importante apparaît entre les deux *backends*, aussi bien pour le calcul que

pour la mémoire. En B0, le GPU atteint 9,62 TFLOP/s, soit 94,3 % de son potentiel FP16 théorique, contre 97,5 GFLOP/s et 56,7 % sur CPU. De même, il exploite environ 87 % du plafond de la DRAM partagée, contre seulement 23 % sur CPU (88,97 contre 23,37 GB/s). Ces fractions restent globalement cohérentes entre les configurations, avec environ 93–95 % du potentiel de calcul atteint sur GPU contre 55–57 % sur CPU.
Cette différence d'exploitation amplifie donc l'asymétrie matérielle. En B0, si CPU et GPU atteignaient tous deux leur pic FP16 théorique, leur rapport de capacité de calcul serait d'environ 59,3, contre 98,7 entre les plafonds empiriques, soit un écart inférieur d'environ 40 %. Côté mémoire, la DRAM étant physiquement partagée, le rapport de 3,81 entre les bandes passantes mesurées provient principalement de la fraction très différente du sous-système mémoire effectivement exploitée par chaque *backend*. L'écart CPU/GPU peut ainsi refléter à la fois les capacités matérielles disponibles et leur exploitation par le chemin d'exécution considéré.

4.2.1 Influence des paramètres système

Les Figures 4.1(c) à (f) permettent ensuite d'expliquer comment les variations OFAT déplacent les rapports précédents. Elles montrent directement les plafonds de calcul et de bande passante, puis les coefficients énergétiques ϵ_f et ϵ_m mesurés sous chaque configuration.

Du côté des performances, une première tendance apparaît clairement : les paramètres CPU, `CPUF` et `CPUC`, modifient principalement les capacités du CPU, tandis que `GPUF` et `GPUT` agissent principalement sur celles du GPU. `EMCF` se distingue puisqu'il agit sur la mémoire partagée et affecte ainsi la bande passante accessible aux deux *backends*. Du côté énergétique, les variations sont moins directement localisées : les coefficients ϵ_f et ϵ_m peuvent évoluer même lorsque le paramètre modifié cible principalement l'autre *backend*, ce qui est cohérent avec le caractère intégré du SoC et avec une mesure énergétique effectuée à l'échelle de la plateforme.

Le rapprochement le plus large est obtenu avec `CPUC_HIGH`. Le plafond de calcul CPU passe de 97,48 à 147,08 GFLOP/s, soit une hausse de 50,9 %, tandis que sa bande passante passe de 23,37 à 36,18 GB/s, soit +54,8 %. Les capacités GPU restent, elles, pratiquement inchangées. L'ajout de cœurs agit donc simultanément sur les deux composantes du *Roofline* CPU et produit le rapprochement le plus important sur pratiquement toute la plage d'intensité, comme le montre la Figure 4.1(a). En performance, le rapport GPU/CPU passe ainsi de 3,81 à 2,46 dans la région mémoire et de 98,7 à 65,5 dans la région calcul. Le rapprochement apparaît également sur l'efficacité énergétique : R_E passe de 2,55 à 1,99 à $I = 0{,}7$ FLOP/B et de 44,6 à 33,8 à $I = 250$ FLOP/B. Les coefficients énergétiques CPU évoluent dans le même sens, avec ϵ_f qui passe d'environ 26,68 à 25,14 pJ/FLOP et ϵ_m de 170,83 à 163,01 pJ/B.

La fréquence CPU produit un effet plus localisé. Avec `CPUF_HIGH`, le plafond de calcul CPU passe de 97,48 à 123,92 GFLOP/s, soit +27,1 %, alors que la bande passante n'augmente que de 3,4 %. Son influence sur l'écart apparaît donc surtout à forte intensité : le rapport des plafonds de calcul passe de 98,7 à 78,1, tandis que le rapport des bandes passantes ne passe que de 3,81 à 3,71. La même localisation apparaît dans R_E, qui reste presque inchangé à $I = 0{,}7$ (2,55 contre 2,51), mais diminue de 44,6 à 37,8 à $I = 250$. Les coefficients ϵ_f et ϵ_m du CPU augmentent pourtant respectivement d'environ 9,4 % et 5,5 % ; la réduction de l'écart énergétique à forte intensité provient donc surtout de l'augmentation du plafond de calcul, qui réduit la contribution de la puissance de fond par FLOP dans le modèle énergétique.

Le nombre de TPC agit principalement sur la capacité de calcul GPU. Avec `GPUT_LOW`, le *compute roof* passe de 9,62 à 6,34 TFLOP/s, soit une diminution de 34,1 %, tandis que la bande passante reste pratiquement inchangée autour de 89 GB/s. À faible intensité, l'écart évolue donc peu : R_P reste proche de B0 (3,83 contre 3,81) et R_E reste autour de 2,55. À forte intensité, en revanche, les deux *backends* se rapprochent davantage : à $I = 250$ FLOP/B, R_P passe de 98,7 à 65,9 et R_E de 44,6 à 36,4. Cette évolution énergétique est cohérente avec la hausse de ϵ_f GPU, d'environ 0,99 à 1,13 pJ/FLOP, alors que ϵ_m varie peu.

La fréquence GPU a un effet plus étendu puisqu'elle agit à la fois sur le calcul et sur la bande passante effectivement atteinte. Avec `GPUF_LOW`, le plafond de calcul passe de 9,62 à 7,26 TFLOP/s et la bande passante de 88,97 à 67,20 GB/s, soit une diminution d'environ 24,5 % dans les deux cas. L'écart se réduit ainsi aussi bien à faible qu'à forte intensité : R_P passe de 3,81 à 2,97 dans la région mémoire et de 98,7 à 75,7 dans la région calcul. L'écart énergétique diminue également, de 2,55 à 2,21 à $I = 0{,}7$ et de 44,6 à 38,9 à $I = 250$. Sur l'*Energy Roofline* GPU, ϵ_f reste proche de sa valeur B0 tandis que ϵ_m augmente d'environ 4,1 % ; la réduction de R_E est donc cohérente avec la baisse des plafonds de performance du GPU ; elle ne résulte pas d'une amélioration de ses coûts énergétiques unitaires. L'effet de la fréquence GPU sur la bande passante rejoint l'observation de Prashanthi et al. [29], qui montrent sur AGX Orin que la fréquence GPU n'agit pas uniquement sur le débit de calcul mais également sur la bande passante effectivement atteinte.

Enfin, la fréquence EMC agit principalement sur la bande passante et touche les deux *backends* puisqu'ils partagent la même DRAM. Avec `EMCF_LOW`, la bande passante diminue de 25,9 % sur CPU, de 23,37 à 17,32 GB/s, et de 28,5 % sur GPU, de 88,97 à 63,60 GB/s, alors que leurs plafonds de calcul changent peu. Cette diminution simultanée explique pourquoi l'écart relatif de performance varie peu : dans la région mémoire, R_P passe seulement de 3,81 à 3,67. L'effet énergétique n'est toutefois pas identique. Les coefficients ϵ_f et ϵ_m diminuent sur les deux *backends*, mais ces variations relatives ne sont pas les mêmes côté CPU et côté GPU. À très faible intensité, R_E augmente ainsi légèrement de 2,55 à 2,62, alors qu'à $I \simeq 70$ FLOP/B il diminue d'environ 26,6 à 23,8. Une variation de l'EMC peut modifier différemment l'écart temporel et énergétique selon la région considérée.

4.2.2 Réponse à Q1

L'écart entre CPU et GPU est principalement déterminé par l'intensité arithmétique et par la ressource modifiée par la configuration système. Il est le plus faible à basse intensité, où les deux *backends* dépendent de leur capacité à exploiter la mémoire partagée, puis augmente lorsque le calcul devient dominant. Sur la plateforme et avec le chemin d'exécution étudiés, le GPU conserve néanmoins des plafonds supérieurs au CPU sur toute la plage considérée, aussi bien en performance qu'en efficacité énergétique. L'écart observé ne s'explique pas uniquement par les caractéristiques matérielles : le GPU atteint environ 93–95 % de son potentiel FP16 théorique, contre généralement 55–57 % sur CPU.

Parmi les paramètres étudiés, `CPUC_HIGH` rapproche le plus globalement CPU et GPU. `CPUF_HIGH` et `GPUT_LOW` modifient surtout l'écart à forte intensité, puisqu'ils agissent principalement sur la capacité de calcul, tandis que `GPUF_LOW` le réduit sur une plage plus large en affectant à la fois le calcul et la bande passante GPU. La variation de l'EMC modifie davantage les capacités absolues que l'écart relatif de performance, puisqu'elle agit sur une

ressource partagée ; son effet sur l'écart énergétique dépend en revanche de l'intensité arithmétique.

4.3 Q2 – Transposition des écarts CPU/GPU des Rooflines aux charges de travail LLM

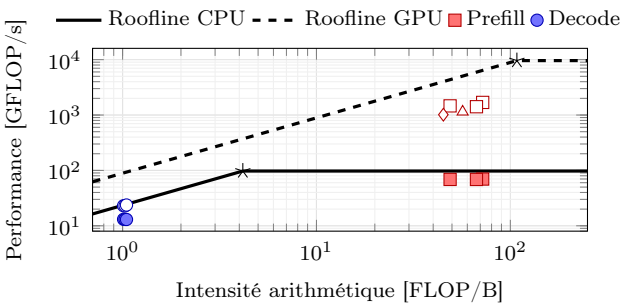
La deuxième question de recherche vise à déterminer dans quelle mesure les écarts CPU/GPU mis en évidence par les *Rooflines* se retrouvent lors de l'exécution réelle des phases de *prefill* et de *decode* positionnées sur ces *Rooflines*, et comment ces écarts évoluent selon les configurations système.

Pour chaque charge de travail, on conserve les rapports R_P et R_E définis en Q1 et on leur associe leurs équivalents applicatifs :

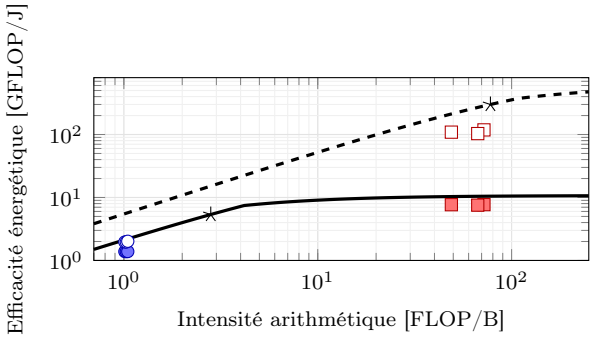
$$R_P^{\mathrm{LLM}} = \frac{P_{\mathrm{GPU}}^{\mathrm{LLM}}}{P_{\mathrm{CPU}}^{\mathrm{LLM}}}, \qquad R_E^{\mathrm{LLM}} = \frac{\eta_{\mathrm{GPU}}^{\mathrm{LLM}}}{\eta_{\mathrm{CPU}}^{\mathrm{LLM}}}. \tag{4.3}$$

où $P^{\mathrm{LLM}} = W/t$ représente la performance applicative et $\eta^{\mathrm{LLM}} = W/E$ son efficacité énergétique.

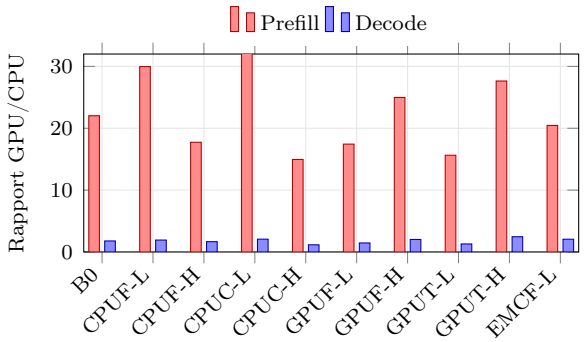
La Figure 4.3 présente d'abord le positionnement des charges de travail sur les *Time Rooflines* et *Energy Rooflines* de la configuration B0, puis, pour chaque configuration, l'écart GPU/CPU moyen associé aux phases de *prefill* et de *decode*. Les valeurs sont moyennées sur les tailles 128, 512 et 1024 *tokens* de chaque phase, et le rapport *Roofline* est évalué à l'intensité arithmétique de chaque charge de travail avant d'être moyenné de la même manière.



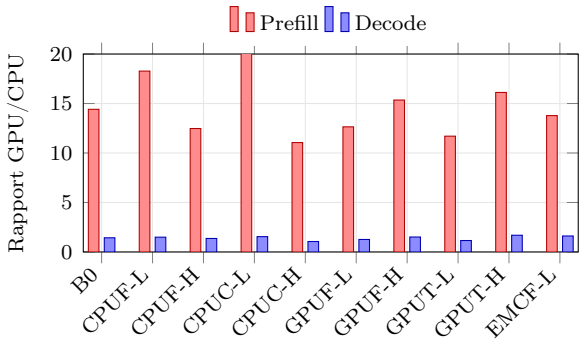
(a) *Time Roofline* en B0. Points pleins : CPU ; points creux : GPU.



(b) *Energy Roofline* en B0 (Alumet), campagne initiale.



(c) Écart applicatif en performance.



(d) Écart applicatif en efficacité énergétique.

FIGURE 4.3 – Positionnement des charges LLM sur les Rooflines et comparaison des écarts CPU/GPU.

La première observation concerne le positionnement des charges de travail. Les Figures 4.3(a) et (b) montrent qu'en B0, le *prefill* se situe du côté calcul sur CPU, tandis que le *decode* se situe du côté mémoire. Sur GPU, les deux phases se placent du côté mémoire, aussi bien dans le *Time Roofline* que dans l'*Energy Roofline*. Ce positionnement reste globalement le même dans les autres configurations.

Le positionnement du *prefill* GPU dépend toutefois du chemin logiciel. Les travaux existants [28] le caractérisent comme *compute-intensive* sur leurs GPU, mais dans notre protocole, Flash Attention est désactivé afin de conserver un graphe logique comparable entre CPU et GPU, ce qui supprime les optimisations mémoire qui peuvent le ramener vers la région de calcul. Dans notre configuration, même avec une taille P4096, le *prefill* reste du côté mémoire, alors qu'un test avec Flash Attention activé fait basculer P1024 du côté calcul. Ce test permet toutefois seulement de conclure que l'exécution se rapproche du point de basculement, car notre estimation logique sous-estime le trafic mémoire mesuré, comme montré dans la partie consacrée à la validation de la caractérisation de la charge.

La deuxième observation, visible dans les Figures 4.3(**??**) à (**??**), est que les écarts applicatifs suivent globalement les variations des *Rooflines*. Les lignes pointillées prolongent les deux valeurs B0 et permettent de lire si une configuration rapproche ou éloigne CPU et GPU par rapport à cette référence. L'intérêt ici n'est pas de réinterpréter l'effet matériel de chaque paramètre, déjà établi en Q1, mais de vérifier si sa conséquence sur l'écart CPU/GPU se transpose à la charge de travail réelle.

`CPUC_HIGH` illustre particulièrement bien cette transposition. Pour le *prefill*, l'écart de performance donné par le *Roofline* diminue de 33,6 % par rapport à B0, tandis que l'écart mesuré sur le LLM diminue de 32,1 %. Sur le *decode*, les diminutions sont respectivement de 35,3 % et 34,9 %. Le même accord apparaît pour l'efficacité énergétique du *prefill*, avec −23,9 % sur le *Roofline* et −23,3 % sur le LLM. `GPUF_LOW` présente la même tendance : l'écart de performance diminue respectivement de 23,2 % et 20,8 % pour le *prefill*, puis de 22,0 % et 18,2 % pour le *decode*. Les *Rooflines* ne décrivent donc pas l'écart applicatif absolu, mais les variations observées sur les charges de travail suivent globalement celles des écarts entre les plafonds, et dans plusieurs cas avec une amplitude relative proche.

Deux configurations s'écartent davantage de cette correspondance. Avec `GPUT`, l'écart *Roofline* varie peu alors que l'écart applicatif change fortement. Pour le *prefill*, le rapport *Roofline* reste proche de B0 (57,7 avec `GPUT_LOW`, 57,1 en B0 et 58,9 avec `GPUT_HIGH`), tandis que le rapport LLM passe de 15,6 à 22,0 puis 27,6. Le même phénomène apparaît sur le *decode*. Ce cas montre ainsi que l'évolution du plafond *Roofline* ne suffit pas toujours à refléter celle des performances applicatives.
`EMCF_LOW` constitue un second cas notable. Sur le *prefill*, l'écart de performance *Roofline* diminue de 26,3 %, mais l'écart LLM seulement de 7,1 %. Sur le *decode*, les deux évoluent même en sens opposé : le rapport *Roofline* diminue de 3,6 %, alors que le rapport applicatif augmente de 15,7 %. Le changement de plafond ne suffit donc pas ici à décrire comment chaque *backend* exploite la nouvelle capacité disponible.

La troisième observation est commune à tous les panneaux : l'écart GPU/CPU mesuré sur le LLM est nettement inférieur à celui observé entre les *Rooflines*. Cette réduction s'explique par la fraction différente du plafond atteinte par chaque *backend*. En B0, le CPU atteint en moyenne 70,9 % de son *Time Roofline* pour le *prefill* et 54,2 % pour le *decode*, contre seulement 27,9 % et 25,4 % sur GPU. La même tendance apparaît sur l'*Energy Roofline* : 73,4 % et 64,0 %

côté CPU, contre 43,9 % et 35,3 % côté GPU. Les charges de travail CPU exploitent donc une fraction plus élevée du plafond empirique fourni par le chemin `llama.cpp/ggml`, ce qui réduit l'avantage GPU observé en Q1. En B0, le rapport moyen de performance du *prefill* passe ainsi de 57,1 entre les *Rooflines* à 22,0 sur le LLM ; pour le *decode*, il passe de 3,81 à 1,78.

Cette réduction est particulièrement marquée dans la région de faible intensité occupée par le *decode*. Avec `CPUC_HIGH`, le rapport moyen n'est plus que de 1,16 en performance et 1,06 en efficacité énergétique : CPU et GPU deviennent ainsi presque équivalents sur le plan énergétique, sans qu'un *crossover* soit observé. Ce résultat fait écho à Zhang et Huang [42], qui montrent sur iPhone 15 Pro qu'un CPU peut dépasser le GPU pour des phases de *prefill* et de génération de petite taille. Ces deux résultats suggèrent ainsi que les charges de travail dominées par la génération et de faible intensité constituent une région où le CPU peut devenir plus compétitif, l'apparition d'un *crossover* dépendant ensuite du matériel et de l'efficacité des chemins logiciels.

4.3.1 Réponse à Q2

Les écarts CPU/GPU observés sur les charges de travail LLM suivent globalement les variations des *Rooflines*, même si certains paramètres comme `GPUT` et `EMCF` s'en écartent davantage. L'écart applicatif reste toutefois nettement plus faible, car les charges de travail CPU exploitent une fraction plus importante de leur plafond que les charges de travail GPU. Ce rapprochement est particulièrement marqué sur le *decode*, jusqu'à une quasi-parité énergétique sous `CPUC_HIGH`. Les *Rooflines* permettent ainsi de caractériser la tendance générale de l'écart CPU/GPU, tandis que son amplitude réelle dépend de l'efficacité des *kernels* et du chemin logiciel utilisé.

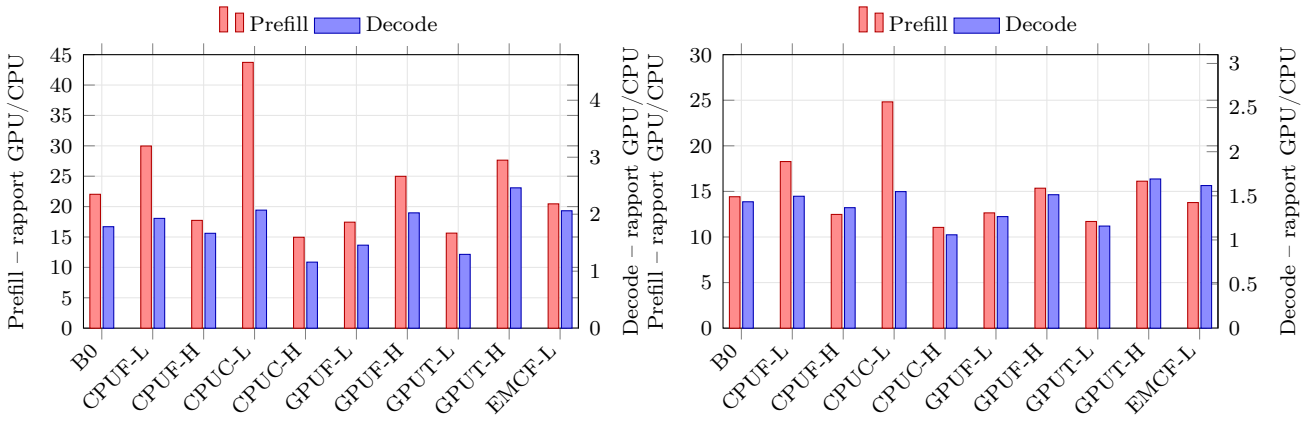
4.4 Q3 – Influence de l'instrument de mesure sur les conclusions énergétiques

La troisième question de recherche vise à déterminer si Alumet et le Power Analyzer conduisent aux mêmes conclusions lorsque l'on compare les configurations système.

Une première différence apparaît sur les valeurs absolues. Sur l'ensemble des exécutions LLM mesurées, l'énergie rapportée par Alumet est systématiquement inférieure à celle du Power Analyzer, avec un écart compris entre 18,7 % et 25,6 %, et de 21,2 % en moyenne. Notre protocole ne permet toutefois pas d'identifier expérimentalement l'origine de cet écart, qui peut provenir, par exemple, d'une différence de périmètre de mesure entre `VDD_IN` pour Alumet et l'alimentation complète de la carte pour le Power Analyzer.

Cette hypothèse ne pouvant pas être vérifiée sans une configuration de mesure spécifique, l'analyse porte principalement sur sa conséquence, à savoir si les deux instruments reproduisent les mêmes variations entre configurations et conduisent aux mêmes choix.

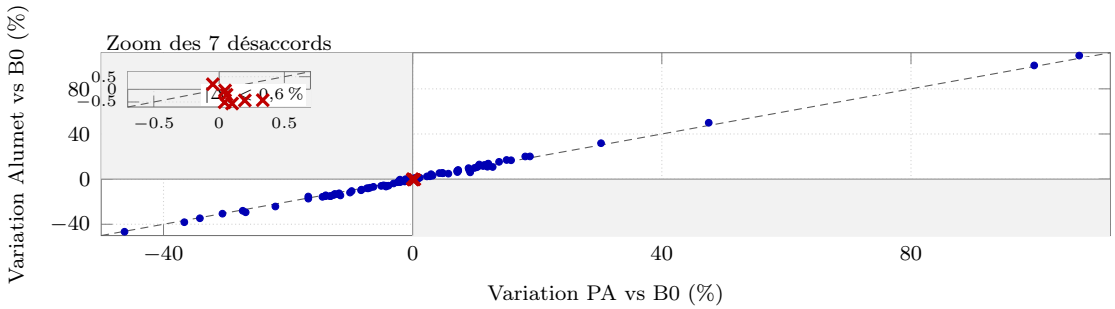
La Figure 4.5 synthétise cette comparaison. Le panneau (a) compare les variations d'énergie par rapport à B0 et met en évidence les éventuels désaccords sur leur sens, tandis que le panneau (b) compare le classement des configurations pour chacune des six charges de travail et des deux *backends*.



(a) Écart GPU/CPU en performance.

(b) Écart GPU/CPU en efficacité énergétique.

FIGURE 4.4 – Évolution de l'écart GPU/CPU des phases de *prefill* et de *decode* selon les configurations système.



(a) Variations relatives à B0.

	P128	P512	P1024	O128	O512	O1024
CPU	1,000 •	1,000 •	0,976 ×	0,988 •	0,939 •	0,976 •
GPU	0,988 •	0,933 •	1,000 •	0,964 •	0,976 •	1,000 •

ρ_S moyen = 0,978 • même optimum × optimum différent 11/12 mêmes optima

(b) Accord sur le classement des configurations.

FIGURE 4.5 – Comparaison des mesures énergétiques d'Alumet et du Power Analyzer.

La première comparaison porte sur les variations d'énergie par rapport à B0, calculées pour un même *backend* et une même charge de travail. Après filtrage des groupes exploitables, 114 groupes de mesures sont retenus. Parmi eux, les 12 groupes B0, correspondant aux six charges de travail et aux deux *backends*, servent de références, ce qui laisse 102 comparaisons relatives. Dans la Figure 4.5(a), l'écart entre les variations mesurées par les deux instruments reste faible sur l'ensemble des cas : leur coefficient de corrélation de Pearson atteint 0,999 et leur écart absolu moyen n'est que de 0,91 point de pourcentage.

Les deux instruments indiquent également le même sens d'évolution dans 95 cas sur 102. Les sept désaccords, représentés par des croix, sont tous concentrés autour de variations pratiquement nulles : elles restent inférieures à 0,34 % avec le Power Analyzer et à 0,6 % avec Alumet, et correspondent donc à des configurations presque équivalentes à B0. Malgré l'écart observé sur les valeurs absolues, les variations induites par les configurations système sont ainsi reproduites de manière très proche par les deux instruments.

La seconde comparaison porte sur l'ordre des configurations pour chacune des six charges de travail et des deux *backends*. La Figure 4.5(b) montre que les corrélations de rang de Spearman sont comprises entre 0,933 et 1, avec une moyenne de 0,978. Les différences entre instruments se limitent donc principalement à quelques permutations entre configurations proches.

Cette concordance se retrouve dans le choix de la configuration la plus économe : Alumet et le Power Analyzer identifient le même optimum dans 11 des 12 couples *backend*-charge de travail. Le seul désaccord apparaît pour le *prefill* P1024 sur CPU, où Alumet place `EMCF_LOW` devant `CPUC_LOW`, tandis que le Power Analyzer inverse ces deux configurations. L'écart reste cependant inférieur à 1 % : choisir `EMCF_LOW` à partir d'Alumet entraîne seulement 0,92 % d'énergie supplémentaire selon le Power Analyzer. Le choix de l'instrument peut donc modifier l'ordre de configurations presque équivalentes, sans remettre en cause les différences énergétiques observées dans la campagne ; dans un cas plus sensible, cette différence devrait toutefois être considérée avec prudence.

4.4.1 Réponse à Q3

Alumet et le Power Analyzer ne fournissent pas les mêmes valeurs énergétiques absolues, mais ils conduisent à des conclusions très proches pour comparer les configurations. Les variations par rapport à B0 sont fortement corrélées, et les rares désaccords de direction concernent uniquement des effets proches de zéro. De même, les classements sont très similaires, et la meilleure configuration est identique dans 11 cas sur 12, l'unique différence conduisant à un écart inférieur à 1 %. Dans cette campagne, le choix de l'instrument affecte donc principalement la valeur absolue rapportée, mais très peu l'analyse comparative et le choix des configurations.

4.5 Synthèse du chapitre

Les trois questions de recherche mettent en évidence des résultats complémentaires. Q1 montre que l'écart CPU/GPU dépend fortement de l'intensité arithmétique et des ressources modifiées par la configuration : il est minimal dans la région mémoire et `CPUC_HIGH` produit

le rapprochement le plus important. Q2 montre que ces variations se retrouvent globalement sur les charges de travail LLM, mais avec des écarts applicatifs nettement plus faibles, en particulier pour le *decode*, où CPU et GPU s'approchent d'une efficacité équivalente. Enfin, Q3 montre que, malgré un décalage sur les valeurs énergétiques absolues, Alumet et le Power Analyzer conduisent à des variations et à des classements très proches des configurations.

4.6 Conclusion

Ce chapitre a permis de répondre aux trois questions de recherche en reliant les écarts CPU/GPU à l'intensité arithmétique, aux configurations système et à l'exploitation effective des plafonds, puis en évaluant la cohérence des deux méthodes de mesure énergétique. Ces résultats constituent la base de la conclusion générale et des perspectives présentées dans la suite du mémoire.

Conclusion générale

Ce mémoire présente le travail réalisé à ENSTA Campus Brest, au sein du Lab-STICC, autour de la caractérisation des performances et de la consommation énergétique de l'inférence des LLMs sur une plateforme *Edge*. L'objectif principal était d'étudier comment cette consommation évolue selon la charge, la ressource d'exécution et les paramètres système, puis de proposer une modélisation permettant d'interpréter les différences observées entre CPU et GPU.

Un état de l'art a d'abord permis d'identifier les principaux facteurs étudiés dans les travaux existants ainsi que les approches de caractérisation et d'optimisation énergétique. À partir de cette analyse, une méthodologie basée sur les *Time Rooflines* et les *Energy Rooflines* a été mise en place. Les capacités du CPU et du GPU ont été caractérisées sous différentes configurations système, puis des charges LLM de *prefill* et de *decode* ont été positionnées sur ces modèles. Deux méthodes de mesure énergétique, Alumet et un analyseur de puissance externe, ont également été comparées.

Les résultats montrent que, sur la plateforme étudiée, le GPU conserve des capacités supérieures au CPU, mais que l'écart dépend fortement de l'intensité arithmétique et de la configuration système. Il est plus faible pour les charges dominées par les accès mémoire et se réduit notamment lorsque le nombre de cœurs CPU augmente. Les mesures applicatives suivent globalement les variations observées sur les *Rooflines*, avec toutefois des écarts CPU/GPU plus faibles, en particulier pour le *decode*, où une quasi-parité énergétique est observée dans certaines configurations. Enfin, malgré un écart sur les valeurs énergétiques absolues, les deux instruments conduisent à des variations relatives et à des classements très proches.

Difficultés rencontrées

La première difficulté a été de délimiter précisément le périmètre de l'étude, notamment de déterminer jusqu'où approfondir la caractérisation avant d'envisager la mise en œuvre de stratégies d'optimisation. Le travail a finalement été recentré sur la caractérisation et la modélisation, afin de disposer d'une compréhension suffisamment fine des comportements observés avant d'aborder leur optimisation.

Une seconde difficulté a été de délimiter un espace expérimental comportant de nombreux facteurs applicatifs, système et matériels, tout en conservant un nombre d'expériences réalisable. La construction des *Energy Rooflines* a également nécessité un protocole stable et reproductible, ainsi qu'un contrôle rigoureux des configurations et des conditions de mesure.

Enfin, l'estimation de l'intensité arithmétique des charges LLM a constitué une difficulté

particulière. Elle a nécessité l'extraction et l'analyse des graphes d'exécution de `llama.cpp`, ainsi qu'une distinction entre le volume logique manipulé par les opérations et le trafic physique observé en mémoire.

Limites de l'étude

Plusieurs limites doivent être soulignées :

- l'étude principale repose sur une seule plateforme NVIDIA Jetson Orin Nano Super, ce qui limite la généralisation des résultats à d'autres architectures *Edge* ;
- un seul modèle, un seul moteur d'inférence et un format FP16 sont utilisés pour la caractérisation applicative ;
- l'approche OFAT permet d'isoler l'effet des paramètres système, mais ne permet pas d'étudier leurs interactions lorsqu'ils varient simultanément ;

Perspectives

Une première perspective consiste à étendre la caractérisation à d'autres modèles, moteurs, formats numériques et plateformes afin d'évaluer la généralisation des tendances observées.

Cette caractérisation pourrait ensuite être exploitée pour développer une stratégie d'optimisation capable de choisir ou d'adapter la ressource d'exécution et la configuration système selon la phase de l'inférence, la charge et les contraintes de performance ou d'énergie. Elle pourrait également permettre d'étudier des stratégies de déport partiel de l'exécution, dans lesquelles certaines parties du modèle seraient exécutées sur le CPU lorsque cette ressource présente un meilleur compromis énergétique ou lorsque le GPU doit être réservé à d'autres charges.

Bibliographie

[1] Maximilian Abstreiter, Sasu Tarkoma et Roberto Morabito. « Sometimes Painful but Promising : Feasibility and Trade-Offs of On-Device Language Model Inference ». In : *ACM Transactions on Embedded Computing Systems* 25.2 (2026), p. 1-30. doi : 10.1145/3788870.

[2] Andrea Apicella et al. « Energy Consumption Assessment in Embedded AI : Metrological Improvements of Benchmarks for Edge Devices ». In : *Computer Standards & Interfaces* 97 (2026), p. 104120. doi : 10.1016/j.csi.2025.104120.

[3] Mauricio Fadel Argerich, Jonathan Fürst et Marta Patiño-Martínez. « WattGPU : Predicting Inference Power and Latency on Unseen GPUs and LLMs ». In : *arXiv preprint arXiv :2607.02391* (2026). doi : 10.48550/arXiv.2607.02391. arXiv : 2607.02391.

[4] Mayank Arya et Yogesh Simmhan. « Understanding the Performance and Power of LLM Inferencing on Edge Accelerators ». In : *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 2025, p. 1108-1111. doi : 10.1109/IPDPSW66978.2025.00173.

[5] Loubna Ben Allal et al. « SmolLM2 : When Smol Goes Big – Data-Centric Training of a Small Language Model ». In : *arXiv preprint arXiv :2502.02737* (2025). doi : 10.48550/arXiv.2502.02737. arXiv : 2502.02737.

[6] Tom B. Brown et al. « Language Models are Few-Shot Learners ». In : *Advances in Neural Information Processing Systems*. T. 33. 2020, p. 1877-1901.

[7] Francisco Caravaca, Ángel Cuevas et Rubén Cuevas. « From Prompts to Power : Measuring the Energy Footprint of LLM Inference ». In : *arXiv preprint arXiv :2511.05597* (2025). doi : 10.48550/arXiv.2511.05597. arXiv : 2511.05597.

[8] Jiesong Chen et al. « Energy-Efficient Small Language Model Inference for Mobile Agents via DVFS ». In : *Proceedings of the 24th Annual International Conference on Mobile Systems, Applications and Services Workshops*. Association for Computing Machinery, 2026, p. 182-187. doi : 10.1145/3812836.3814753.

[9] Jared Fernandez et al. « Energy Considerations of Large Language Model Inference and Efficiency Optimizations ». In : *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1 : Long Papers)*. Vienna, Austria : Association for Computational Linguistics, juill. 2025, p. 32556-32569. doi : 10.18653/v1/2025.acl-long.1563.

[10] Guillaume FIENI et al. « PowerAPI : A Python framework for building software-defined power meters ». In : *Journal of Open Source Software* 9.98 (2024), p. 6670. DOI : 10.21105/joss.06670.

[11] Georgi GERGANOV et CONTRIBUTORS. *llama.cpp : LLM Inference in C/C++*. Open-source software project, ggml-org. 2023.

[12] Henrikas GIEDRA et al. « Benchmarking Large Language Model Inference on Limited-Resource Edge Systems ». In : *Electronics* 15.11 (2026), p. 2451. DOI : 10.3390/electronics15112451.

[13] Sanjay Surendranath GIRIJA et al. « Optimizing LLMs for Resource-Constrained Environments : A Survey of Model Compression Techniques ». In : *arXiv preprint arXiv :2505.02309* (2025). DOI : 10.48550/arXiv.2505.02309. arXiv : 2505.02309.

[14] Erik Johannes HUSOM et al. « Sustainable LLM Inference for Edge AI : Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency ». In : *ACM Transactions on Internet of Things* 6.4 (2025), p. 1-35. DOI : 10.1145/3767742.

[15] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Software Developer's Manual*. Power and Thermal Management ; Running Average Power Limit (RAPL). Intel Corporation. 2026.

[16] INTERNATIONAL ENERGY AGENCY. *Energy and AI*. Rapp. tech. Paris : International Energy Agency, avr. 2025. URL : https://www.iea.org/reports/energy-and-ai.

[17] Md. Romyull ISLAM et al. « Characterizing and Understanding Energy Footprint and Efficiency of Small Language Model on Edges ». In : *2025 IEEE 22nd International Conference on Mobile Ad-Hoc and Smart Systems (MASS)*. 2025, p. 363-371. DOI : 10.1109/MASS66014.2025.00059.

[18] Javier JAREÑO et al. « Energy-Efficient Large Language Models ». In : *Future Generation Computer Systems* 182 (2026), p. 108483. DOI : 10.1016/j.future.2026.108483.

[19] Mathilde JAY et al. « An Experimental Comparison of Software-Based Power Meters : Focus on CPU and GPU ». In : *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. 2023, p. 106-118. DOI : 10.1109/CCGrid57682.2023.00020.

[20] Jared KAPLAN et al. « Scaling Laws for Neural Language Models ». In : *arXiv preprint arXiv :2001.08361* (2020). DOI : 10.48550/arXiv.2001.08361. arXiv : 2001.08361.

[21] Seyyidahmed LAHMER et al. « Energy Consumption of Neural Networks on NVIDIA Edge Boards : An Empirical Model ». In : *2022 20th International Symposium on Modeling and Optimization in Mobile, Ad hoc, and Wireless Networks (WiOpt)*. 2022, p. 365-371. DOI : 10.23919/WiOpt56218.2022.9930584.

[22] Paul Joe MALIAKEL, Shashikant ILAGER et Ivona BRANDIC. « Investigating Energy Efficiency and Performance Trade-offs in LLM Inference Across Tasks and DVFS Settings ». In : *arXiv preprint arXiv :2501.08219* (2025). DOI : 10.48550/arXiv.2501.08219. arXiv : 2501.08219.

[23] Peter MARWEDEL. *Embedded System Design : Embedded Systems Foundations of Cyber-Physical Systems, and the Internet of Things*. 4e éd. Cham : Springer, 2021. DOI : 10.1007/978-3-030-60910-8.

[24] Mergen NACHIN et al. « ExecuTorch : A Unified PyTorch Solution to Run ML Models On-Device ». In : *Proceedings of Machine Learning and Systems*. T. 8. 2026.

[25] Chien Van NGUYEN et al. « A Survey on Small Language Models ». In : *Proceedings of the 15th International Conference on Recent Advances in Natural Language Processing*. 2025, p. 807-821. DOI : 10.26615/978-954-452-098-4-093.

[26] Chenxu NIU et al. « TokenPowerBench : Benchmarking the Power Consumption of LLM Inference ». In : *Proceedings of the AAAI Conference on Artificial Intelligence*. T. 40. 38. 2026, p. 32582-32590. DOI : 10.1609/aaai.v40i38.40535.

[27] NVIDIA. *NVIDIA Jetson Linux Developer Guide : Platform Power and Performance – Jetson Orin Series*. Documentation on power modes, nvpmodel, DVFS, EMC, TPC configuration and INA3221 power monitors. NVIDIA Corporation. 2026.

[28] Pratyush PATEL et al. « Splitwise : Efficient Generative LLM Inference Using Phase Splitting ». In : *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024, p. 118-132. DOI : 10.1109/ISCA59077.2024.00019.

[29] Prashanthi S. K. et al. « Pagoda : An Energy and Time Roofline Study for DNN Workloads on Edge Accelerators ». In : *arXiv preprint arXiv :2509.20189* (2025). DOI : 10.48550/arXiv.2509.20189. arXiv : 2509.20189.

[30] Siddharth SAMSI et al. « From Words to Watts : Benchmarking the Energy Costs of Large Language Model Inference ». In : *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. 2023, p. 1-9. DOI : 10.1109/HPEC58863.2023.10363447.

[31] Souvika SARKAR et al. « Processing Natural Language on Embedded Devices : How Well Do Transformer Models Perform ? » In : *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*. 2024. DOI : 10.1145/3629526.3645054.

[32] Mahadev SATYANARAYANAN. « The Emergence of Edge Computing ». In : *Computer* 50.1 (2017), p. 30-39. DOI : 10.1109/MC.2017.9.

[33] Sandeep Singh SENGAR et al. « Generative Artificial Intelligence : A Systematic Review and Applications ». In : *Multimedia Tools and Applications* 84.21 (2025), p. 23661-23700. DOI : 10.1007/s11042-024-20016-1.

[34] Neda SHALAVI et al. « Accurate Calibration of Power Measurements from Internal Power Sensors on NVIDIA Jetson Devices ». In : *2023 IEEE International Conference on Edge Computing and Communications (EDGE)*. 2023, p. 166-170. DOI : 10.1109/EDGE60047.2023.00034.

[35] Jovan STOJKOVIC et al. « DynamoLLM : Designing LLM Inference Clusters for Performance and Energy Efficiency ». In : *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 2025, p. 1348-1362. DOI : 10.1109/HPCA61900.2025.00102.

[36] Tina VARTZIOTIS et al. « From Tokens to Watt-hours : Analytical Energy Estimation for LLM Inference on Modern GPUs ». In : *arXiv preprint arXiv :2607.26571* (2026). DOI : 10.48550/arXiv.2607.26571. arXiv : 2607.26571.

[37] Ashish VASWANI et al. « Attention Is All You Need ». In : *Advances in Neural Information Processing Systems*. T. 30. 2017, p. 5998-6008.

[38] Rui WANG et al. « Empowering Large Language Models to Edge Intelligence : A Survey of Edge Efficient LLMs and Techniques ». In : *Computer Science Review* 57 (2025), p. 100755. DOI : 10.1016/j.cosrev.2025.100755.

[39] Grant WILKINS, Srinivasan KESHAV et Richard MORTIER. « Offline Energy-Optimal LLM Serving : Workload-Based Energy Models for LLM Inference on Heterogeneous Systems ». In : *The 3rd ACM HotCarbon Workshop on Sustainable Computer Systems (HotCarbon '24)*. 7 pages. 2024. URL : https://hotcarbon.org/assets/2024/pdf/hotcarbon24-final30.pdf.

[40] Hao XU et al. « Camel : Energy-Aware LLM Inference on Resource-Constrained Devices ». In : *arXiv preprint arXiv :2508.09173* (2025). DOI : 10.48550/arXiv.2508.09173. arXiv : 2508.09173.

[41] Weisi YANG et Stephen XIA. « PELM : Power Efficient On-Device LLM Inference with Speculative Decoding and Dynamic Voltage Frequency Scaling ». In : *Proceedings of the 2026 ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems*. SenSys '26. Association for Computing Machinery, 2026, p. 438-451. DOI : 10.1145/3774906.3802783.

[42] Haolin ZHANG et Jeff HUANG. « Challenging GPU Dominance : When CPUs Outperform for On-Device LLM Inference ». In : *arXiv preprint arXiv :2505.06461* (2025).

[43] Yue ZHENG et al. « A Review on Edge Large Language Models : Design, Execution, and Applications ». In : *ACM Computing Surveys* 57.8 (2025), p. 1-35. DOI : 10.1145/3719664.

[44] Bohua ZOU et al. « EnerInfer : Energy-Aware On-Device LLM Inference ». In : *arXiv preprint arXiv :2606.23001* (2026). DOI : 10.48550/arXiv.2606.23001. arXiv : 2606.23001.